Bitcoin Utilities Documentation

Release 0.8.4

Konstantinos Karasavvas

May 18, 2026

USER GUIDE

`python-bitcoin-utils` is a pure-Python educational library for building, parsing, and signing Bitcoin data structures. It exposes low-level objects for keys, addresses, scripts, transactions, PSBTs, blocks, HD derivation, and Bitcoin Core RPC calls.

The library is intentionally close to the Bitcoin primitives. You usually build objects directly, inspect their fields, serialize them, and pass them to the next step. This documentation is organized the same way: start with workflows, then use the API reference for exact methods and arguments.

CHAPTER

ONE

SECURITY MODEL

The private-key code is pure Python and intended for learning, tests, testnet, and offline experimentation. ECDSA signing and Taproot/Schnorr signing are not side-channel hardened. Do not use this library to protect real funds in timing-observable environments.

On mainnet, the library emits a one-time warning when private-key operations are used. Testnet, testnet4, signet, and regtest stay quiet so examples remain usable in teaching material.

CHAPTER

TWO

GETTING AROUND

2.1 Quick Start

Every script should first select a network:

```
from bitcoinutils.setup import setup

setup("testnet")
```

2.1.1 Keys and Addresses

Create a private key, derive its public key, and render common address types:

```
from bitcoinutils.keys import PrivateKey

private_key = PrivateKey()
public_key = private_key.get_public_key()

print(private_key.to_wif())
print(public_key.to_hex())
print(public_key.get_address().to_string())
print(public_key.get_segwit_address().to_string())
print(public_key.get_taproot_address().to_string())
```

2.1.2 Simple P2PKH Transaction

The complete P2PKH example builds an unsigned transaction, signs one input, and places the signature and public key in the input scriptSig:

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.


```

(continues on next page)

(continued from previous page)

```
from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.transactions import Transaction, TxInput, TxOutput
from bitcoinutils.keys import P2pkhAddress, PrivateKey
from bitcoinutils.script import Script


def main():
    # always remember to setup the network
    setup("testnet")

    # create transaction input from tx id of UTXO (contained 0.4 tBTC)
    txin = TxInput(
        "fb48f4e23bf6ddf606714141ac78c3e921c8c0bebeb7c8abb2c799e9ff96ce6c", 0
    )

    # create transaction output using P2PKH scriptPubKey (locking script)
    addr = P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
    txout = TxOutput(
        to_satoshis(0.1),
        Script(
            ["OP_DUP", "OP_HASH160", addr.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
        ),
    )

    # create another output to get the change - remaining 0.01 is tx fees
    # note that this time we used to_script_pub_key() to create the P2PKH
    # script
    change_addr = P2pkhAddress("mmYNBho9BWQB2dSniP1NJvnPoj5EVWw89w")
    change_txout = TxOutput(to_satoshis(0.29), change_addr.to_script_pub_key())
    # change_txout = TxOutput(to_satoshis(0.29), Script(['OP_DUP', 'OP_HASH160',
    #                                    change_addr.to_hash160(),
    #                                    'OP_EQUALVERIFY', 'OP_CHECKSIG']))

    # create transaction from inputs/outputs -- default locktime is used
    tx = Transaction([txin], [txout, change_txout])

    # print raw transaction
    print("\nRaw unsigned transaction:\n" + tx.serialize())

    # use the private key corresponding to the address that contains the
    # UTXO we are trying to spend to sign the input
    sk = PrivateKey("cRvyLwCPLU88jsyj94L7iJjQX5C2f8koG4G2gevN4BeSGcEvfKe9")

    # note that we pass the scriptPubkey as one of the inputs of sign_input
    # because it is used to replace the scriptSig of the UTXO we are trying to
    # spend when creating the transaction digest
    from_addr = P2pkhAddress("myPAE9HwPeKHh8FjKwBNBaHnemApo3dw6e")
    sig = sk.sign_input(
        tx,
        0,
        Script(
```

(continues on next page)

(continued from previous page)

```
            [
                "OP_DUP",
                "OP_HASH160",
                from_addr.to_hash160(),
                "OP_EQUALVERIFY",
                "OP_CHECKSIG",
            ]
        ),
    )
    # print(sig)

    # get public key as hex
    pk = sk.get_public_key().to_hex()

    # set the scriptSig (unlocking script)
    txin.script_sig = Script([sig, pk])
    signed_tx = tx.serialize()

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + signed_tx)

    # print the size of the final transaction
    print("\nSigned transaction size (in bytes):\n" + str(tx.get_size()))


if __name__ == "__main__":
    main()
```

2.1.3 Running Examples From the Checkout

If you run examples directly from the repository, use `PYTHONPATH=.` so Python imports the local checkout instead of an installed package:

```
PYTHONPATH=. venv/bin/python examples/p2pkh_transaction.py
```

2.2 Keys

Key objects are defined in `bitcoinutils.keys`.

2.2.1 Private Keys

`PrivateKey` can be created randomly, from WIF, from raw bytes, or from a secret exponent:

```
from bitcoinutils.setup import setup
from bitcoinutils.keys import PrivateKey

setup("testnet")

random_key = PrivateKey()
wif_key = PrivateKey.from_wif("cRvyLwCPLU88jsyj94L7iJjQX5C2f8koG4G2gevN4BeSGcEvfKe9")
exponent_key = PrivateKey(secret_exponent=1)
```

(continues on next page)

(continued from previous page)

```
print(random_key.to_wif())
print(wif_key.to_bytes().hex())
print(exponent_key.get_public_key().to_hex())
```

2.2.2 Public Keys

`PublicKey` accepts compressed SEC, uncompressed SEC, or x-only Taproot-style hex strings.

```
from bitcoinutils.keys import PublicKey

pub = PublicKey("0279be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798")

print(pub.to_hex())
print(pub.to_hex(compressed=False))
print(pub.to_x_only_hex())
print(pub.to_hash160())
```

2.2.3 Message Signing

Bitcoin message signatures use the compact recoverable format. The library can sign and verify them:

```
message = "The test!"
signature = wif_key.sign_message(message)
address = wif_key.get_public_key().get_address().to_string()

assert signature is not None
assert PublicKey.verify_message(address, signature, message)
```

2.2.4 Security Note

Private-key operations are pure Python and are not side-channel hardened. Warnings are emitted on mainnet by default and can be disabled with:

```
from bitcoinutils.setup import set_security_warnings

set_security_warnings(False)
```

2.2.5 Example

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

```

(continues on next page)

(continued from previous page)

```

from bitcoinutils.setup import setup
from bitcoinutils.keys import PrivateKey, PublicKey


def main():
    # always remember to setup the network
    setup("mainnet")

    # create a private key (deterministically)
    priv = PrivateKey(secret_exponent=1)

    # compressed is the default
    print("\nPrivate key WIF:", priv.to_wif(compressed=True))

    # could also instantiate from existing WIF key
    # priv = PrivateKey.from_wif('KwDiBf89qGgbjEhKnhxjUh7LrciVRzI3qYjgd9m7Rfu73SvHnOwn')

    # get the public key
    pub = priv.get_public_key()

    # compressed is the default
    print("Public key:", pub.to_hex(compressed=True))

    # get address from public key
    address = pub.get_address()

    # print the address and hash160 - default is compressed address
    print("Address:", address.to_string())
    print("Hash160:", address.to_hash160())

    print("\n----------------------------------------\n")

    # sign a message with the private key and verify it
    message = "The test!"
    signature = priv.sign_message(message)
    assert signature is not None
    print("The message to sign:", message)
    print("The signature is:", signature)

    if PublicKey.verify_message(address.to_string(), signature, message):
        print("The signature is valid!")
    else:
        print("The signature is NOT valid!")


if __name__ == "__main__":
    main()
```

2.3 Addresses

Address classes live in `bitcoinutils.keys`.

2.3.1 Supported Types

- `P2pkhAddress`: legacy pay-to-public-key-hash.
- `P2shAddress`: pay-to-script-hash.
- `P2wpkhAddress`: native SegWit v0 pay-to-witness-public-key-hash.
- `P2wshAddress`: native SegWit v0 pay-to-witness-script-hash.
- `P2trAddress`: Taproot, SegWit v1.

2.3.2 From Public Keys

```
from bitcoinutils.keys import PrivateKey

priv = PrivateKey()
pub = priv.get_public_key()

legacy = pub.get_address()
segwit = pub.get_segwit_address()
taproot = pub.get_taproot_address()

print(legacy.to_string())
print(segwit.to_string())
print(taproot.to_string())
```

2.3.3 From Scripts

P2SH addresses can be built directly from a redeem script. P2WSH addresses are built from a witness script:

```
from bitcoinutils.keys import P2shAddress, P2wshAddress
from bitcoinutils.script import Script

redeem_script = Script([pub.to_hex(), "OP_CHECKSIG"])

p2sh_addr = P2shAddress.from_script(redeem_script)
p2wsh_addr = P2wshAddress.from_script(redeem_script)

print(p2sh_addr.to_string())
print(p2wsh_addr.to_string())
```

2.3.4 From Existing Address Strings

```
from bitcoinutils.keys import P2pkhAddress, P2wpkhAddress

p2pkh = P2pkhAddress.from_address("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
p2wpkh = P2wpkhAddress.from_address("tb1qxmt9xgewg6mxc4mvnzvrzu4f2v0gy782fydg0w")

print(p2pkh.to_hash160())
print(p2wpkh.to_witness_program())
```

2.3.5 ScriptPubKeys

All address objects expose `to_script_pub_key()`. Use it when constructing a transaction output:

```
from bitcoinutils.transactions import TxOutput
from bitcoinutils.utils import to_satoshis

txout = TxOutput(to_satoshis(0.001), p2pkh.to_script_pub_key())
```

2.3.6 Example

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.


from bitcoinutils.setup import setup
from bitcoinutils.script import Script
from bitcoinutils.keys import P2wpkhAddress, P2wshAddress, P2shAddress, PrivateKey


def main():
    # always remember to setup the network
    setup("testnet")

    # could also instantiate from existing WIF key
    priv = PrivateKey.from_wif("cVdte9ei2xsVjmZSPtyucG43YZgNkmKTqhwiUA8M4Fc3LdPJxPmZ")

    # compressed is the default
    print("\nPrivate key WIF:", priv.to_wif(compressed=True))

    # get the public key
    pub = priv.get_public_key()

    # compressed is the default
    print("Public key:", pub.to_hex(compressed=True))

    # get address from public key
    address = pub.get_segwit_address()

    # print the address and hash - default is compressed address
    print("Native Address:", address.to_string())
    segwit_hash = address.to_witness_program()
    print("Segwit Hash (witness program):", segwit_hash)
    print("Segwit Version:", address.get_type())

```

(continues on next page)

(continued from previous page)

```
    # test to_string
    addr2 = P2wpkhAddress.from_witness_program(segwit_hash)
    print("Created P2wpkhAddress from Segwit Hash and calculate address:")
    print("Native Address:", addr2.to_string())

    #
    # display P2SH-P2WPKH
    #

    # create segwit address
    addr3 = (
        PrivateKey.from_wif("cTmyBsxMQ3vyh4J3jCKYn2Au7AhTKvqeYuxxkinsg6Rz3BBPrYKK")
        .get_public_key()
        .get_segwit_address()
    )
    # wrap in P2SH address
    addr4 = P2shAddress.from_script(addr3.to_script_pub_key())
    print("P2SH(P2WPKH):", addr4.to_string())

    #
    # display P2WSH
    #
    p2wpkh_key = PrivateKey.from_wif(
        "cNn8itYxAng4xR4eMtrPsrPpDpTdVNuw7Jb6kfhFYZ8DLSZBCg37"
    )
    script = Script(
        ["OP_1", p2wpkh_key.get_public_key().to_hex(), "OP_1", "OP_CHECKMULTISIG"]
    )
    p2wsh_addr = P2wshAddress.from_script(script)
    print("P2WSH of P2PK:", p2wsh_addr.to_string())

    #
    # display P2SH-P2WSH
    #
    p2sh_p2wsh_addr = P2shAddress.from_script(p2wsh_addr.to_script_pub_key())
    print("P2SH(P2WSH of P2PK):", p2sh_p2wsh_addr.to_string())


if __name__ == "__main__":
    main()
```

2.4 Scripts

`Script` represents a Bitcoin script as a list of opcode names, integers, and hex data pushes.

2.4.1 Creating Scripts

```
from bitcoinutils.script import Script

p2pkh_script = Script([
    "OP_DUP",
```

(continues on next page)

(continued from previous page)

```
    "OP_HASH160",
    "751e76e8199196d454941c45d1b3a323f1433bd6",
    "OP_EQUALVERIFY",
    "OP_CHECKSIG",
])

print(p2pkh_script.to_hex())
```

2.4.2 Integers and Data Pushes

Small integers `0` through `16` become `OP_0` through `OP_16`. Larger integers are encoded as script numbers. Hex strings are encoded using the smallest valid pushdata opcode.

```
Script([2, "aa" * 33, "bb" * 33, 2, "OP_CHECKMULTISIG"])
```

2.4.3 Parsing and Copying

```
raw = p2pkh_script.to_hex()
parsed = Script.from_raw(raw)
copied = Script.copy(parsed)
```

2.4.4 ScriptPubKey Helpers

Scripts can be wrapped into P2SH or P2WSH scriptPubKeys:

```
redeem_script = Script(["02" + "11" * 32, "OP_CHECKSIG"])

p2sh_script_pubkey = redeem_script.to_p2sh_script_pub_key()
p2wsh_script_pubkey = redeem_script.to_p2wsh_script_pub_key()
```

2.4.5 Script Type Predicates

The module provides helpers for common output patterns:

```
from bitcoinutils.script import is_p2pkh, is_p2sh, is_p2wpkh, is_p2wsh, is_p2tr

assert is_p2pkh(p2pkh_script)
```

2.4.6 Example

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.
```

(continues on next page)

(continued from previous page)

```
11
12
13  from bitcoinutils.setup import setup
14  from bitcoinutils.utils import to_satoshis
15  from bitcoinutils.transactions import Transaction, TxInput, TxOutput
16  from bitcoinutils.keys import P2pkhAddress, PrivateKey
17  from bitcoinutils.script import Script
18
19
20  #
21  # Note that a non-standard transaction can only be included in a block if a
22  # miner agrees with it. For this to work one needs to use a node setup up
23  # for regtest so that you can mine your own blocks; unless you mine your own
24  # testnet/mainnet blocks.
25  # Node's config file requires:
26  #    regtest=1
27  #    acceptnonstdtxn=1
28  #
29  def main():
30      # always remember to setup the network
31      setup("regtest")
32
33      # create transaction input from tx id of UTXO (contained 0.4 tBTC)
34      txin = TxInput(
35          "e2d08a63a540000222d6a92440436375d8b1bc89a2638dc5366833804287c83f", 1
36      )
37
38      # locking script expects 2 numbers that when added equal 5 (silly example)
39      txout = TxOutput(to_satoshis(0.9), Script(["OP_ADD", "OP_5", "OP_EQUAL"]))
40
41      # create another output to get the change - remaining 0.01 is tx fees
42      # note that this time we used to_script_pub_key() to create the P2PKH
43      # script
44      change_addr = P2pkhAddress("mrCDrCybB6J1vRfbwM5hemdJz73FwDBC8r")
45      change_txout = TxOutput(to_satoshis(2), change_addr.to_script_pub_key())
46
47      # create transaction from inputs/outputs -- default locktime is used
48      tx = Transaction([txin], [txout, change_txout])
49
50      # print raw transaction
51      print("\nRaw unsigned transaction:\n" + tx.serialize())
52
53      # use the private key corresponding to the address that contains the
54      # UTXO we are trying to spend to sign the input
55      sk = PrivateKey("cMahea7zqjxrtgAbB7LSGbcQUr1uX1ojuat9jZodMN87JcbXMTcA")
56
57      # note that we pass the scriptPubkey as one of the inputs of sign_input
58      # because it is used to replace the scriptSig of the UTXO we are trying to
59      # spend when creating the transaction digest
60      from_addr = P2pkhAddress("mrCDrCybB6J1vRfbwM5hemdJz73FwDBC8r")
61      sig = sk.sign_input(
62          tx,
```

(continues on next page)

(continued from previous page)

```
        0,
        Script(
            [
                "OP_DUP",
                "OP_HASH160",
                from_addr.to_hash160(),
                "OP_EQUALVERIFY",
                "OP_CHECKSIG",
            ]
        ),
    )
    # print(sig)

    # get public key as hex
    pk = sk.get_public_key()
    pk = pk.to_hex()
    # print (pk)

    # set the scriptSig (unlocking script)
    txin.script_sig = Script([sig, pk])
    signed_tx = tx.serialize()

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + signed_tx)


if __name__ == "__main__":
    main()
```

2.5 Transactions

The transaction module provides low-level transaction objects:

- `TxInput` references a previous output.
- `TxOutput` carries an amount in satoshis and a locking script.
- `TxWitnessInput` stores a witness stack.
- `Transaction` serializes, parses, hashes, and builds signing digests.
- `Sequence` and `Locktime` help construct timelock values.

2.5.1 Constructing a Transaction

```
from bitcoinutils.transactions import Transaction, TxInput, TxOutput
from bitcoinutils.keys import P2pkhAddress
from bitcoinutils.utils import to_satoshis

txin = TxInput("fb48f4e23bf6ddf606714141ac78c3e921c8c0bebeb7c8abb2c799e9ff96ce6c", 0)
txout = TxOutput(to_satoshis(0.1), P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR").to_
↪script_pub_key())
tx = Transaction([txin], [txout])
```

(continues on next page)

(continued from previous page)

```
print(tx.serialize())
```

2.5.2 Legacy Signing

For legacy inputs, pass the previous output's scriptPubKey to `PrivateKey.sign_input` and then place the signature and public key in `script_sig`.

```
sig = private_key.sign_input(tx, 0, previous_script_pubkey)
txin.script_sig = Script([sig, private_key.get_public_key().to_hex()])
```

2.5.3 SegWit Signing

For SegWit inputs, create the transaction with `has_segwit=True`, pass the spent amount to `sign_segwit_input`, and put the signature stack in `TxWitnessInput`.

```
tx = Transaction([txin], [txout], has_segwit=True)
sig = private_key.sign_segwit_input(tx, 0, script_code, amount)
tx.witnesses.append(TxWitnessInput([sig, private_key.get_public_key().to_hex()]))
```

2.5.4 Parsing Transactions

```
parsed = Transaction.from_raw(tx.serialize())
print(parsed.get_txid())
print(parsed.get_size())
```

2.5.5 Timelocks and RBF

Use `Sequence` for relative timelocks and replace-by-fee sequences, and `Locktime` for transaction-level locktime.

```
from bitcoinutils.constants import TYPE_RELATIVE_TIMELOCK
from bitcoinutils.transactions import Sequence

sequence = Sequence(TYPE_RELATIVE_TIMELOCK, 200).for_input_sequence()
txin = TxInput(prev_txid, 0, sequence=sequence)
```

2.5.6 Examples

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.


```

(continues on next page)

(continued from previous page)

```
from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.transactions import Transaction, TxInput, TxOutput
from bitcoinutils.keys import P2pkhAddress, PrivateKey
from bitcoinutils.script import Script


def main():
    # always remember to setup the network
    setup("testnet")

    # create transaction input from tx id of UTXO (contained 0.4 tBTC)
    txin = TxInput(
        "fb48f4e23bf6ddf606714141ac78c3e921c8c0bebeb7c8abb2c799e9ff96ce6c", 0
    )

    # create transaction output using P2PKH scriptPubKey (locking script)
    addr = P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
    txout = TxOutput(
        to_satoshis(0.1),
        Script(
            ["OP_DUP", "OP_HASH160", addr.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
        ),
    )

    # create another output to get the change - remaining 0.01 is tx fees
    # note that this time we used to_script_pub_key() to create the P2PKH
    # script
    change_addr = P2pkhAddress("mmYNBho9BWQB2dSniP1NJvnPoj5EVWw89w")
    change_txout = TxOutput(to_satoshis(0.29), change_addr.to_script_pub_key())
    # change_txout = TxOutput(to_satoshis(0.29), Script(['OP_DUP', 'OP_HASH160',
    #                                    change_addr.to_hash160(),
    #                                    'OP_EQUALVERIFY', 'OP_CHECKSIG']))

    # create transaction from inputs/outputs -- default locktime is used
    tx = Transaction([txin], [txout, change_txout])

    # print raw transaction
    print("\nRaw unsigned transaction:\n" + tx.serialize())

    # use the private key corresponding to the address that contains the
    # UTXO we are trying to spend to sign the input
    sk = PrivateKey("cRvyLwCPLU88jsyj94L7iJjQX5C2f8koG4G2gevN4BeSGcEvfKe9")

    # note that we pass the scriptPubkey as one of the inputs of sign_input
    # because it is used to replace the scriptSig of the UTXO we are trying to
    # spend when creating the transaction digest
    from_addr = P2pkhAddress("myPAE9HwPeKHh8FjKwBNBaHnemApo3dw6e")
    sig = sk.sign_input(
        tx,
        0,
        Script(
```

(continues on next page)

(continued from previous page)

```
            [
                "OP_DUP",
                "OP_HASH160",
                from_addr.to_hash160(),
                "OP_EQUALVERIFY",
                "OP_CHECKSIG",
            ]
        ),
    )
    # print(sig)

    # get public key as hex
    pk = sk.get_public_key().to_hex()

    # set the scriptSig (unlocking script)
    txin.script_sig = Script([sig, pk])
    signed_tx = tx.serialize()

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + signed_tx)

    # print the size of the final transaction
    print("\nSigned transaction size (in bytes):\n" + str(tx.get_size()))


if __name__ == "__main__":
    main()
```

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.


from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.transactions import Transaction, TxInput, TxOutput
from bitcoinutils.keys import P2pkhAddress, PrivateKey
from bitcoinutils.script import Script
from bitcoinutils.constants import SIGHASH_ALL, SIGHASH_SINGLE, SIGHASH_ANYONECANPAY


def main():
    # always remember to setup the network
    setup("testnet")

```

(continues on next page)

(continued from previous page)

```
    # create transaction input from tx id of UTXO (contained 0.39 tBTC)
    # 0.1 tBTC
    txin = TxInput(
        "76464c2b9e2af4d63ef38a77964b3b77e629dddefc5cb9eb1a3645b1608b790f", 0
    )
    # 0.29 tBTC
    txin2 = TxInput(
        "76464c2b9e2af4d63ef38a77964b3b77e629dddefc5cb9eb1a3645b1608b790f", 1
    )

    # create transaction output using P2PKH scriptPubKey (locking script)
    addr = P2pkhAddress("myPAE9HwPeKHh8FjKwBNBaHnemApo3dw6e")
    txout = TxOutput(
        to_satoshis(0.3),
        Script(
            ["OP_DUP", "OP_HASH160", addr.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
        ),
    )

    # create another output to get the change - remaining 0.01 is tx fees
    change_addr = P2pkhAddress("mmYNBho9BWQB2dSniP1NJvnPoj5EVWw89w")
    change_txout = TxOutput(
        to_satoshis(0.08),
        Script(
            [
                "OP_DUP",
                "OP_HASH160",
                change_addr.to_hash160(),
                "OP_EQUALVERIFY",
                "OP_CHECKSIG",
            ]
        ),
    )

    # create transaction from inputs/outputs -- default locktime is used
    tx = Transaction([txin, txin2], [txout, change_txout])

    # print raw transaction
    print("\nRaw unsigned transaction:\n" + tx.serialize())

    #
    # use the private keys corresponding to the addresses that contains the
    # UTXOs we are trying to spend to create the signatures
    #

    sk = PrivateKey("cTALNpTpRbbxTCJ2A5Vq88UxT44w1PE2cYqiB3n4hRvzyCev1Wwo")
    sk2 = PrivateKey("cVf3kGh6552jU2rLaKwXTKq5APHPoZqCP4GQzQirWGHFoHQ9rEVt")

    # we could have derived the addresses from the secret keys
    from_addr = P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
    from_addr2 = P2pkhAddress("mmYNBho9BWQB2dSniP1NJvnPoj5EVWw89w")

```

(continues on next page)

(continued from previous page)

```
    # sign the first input
    sig = sk.sign_input(
        tx,
        0,
        Script(
            [
                "OP_DUP",
                "OP_HASH160",
                from_addr.to_hash160(),
                "OP_EQUALVERIFY",
                "OP_CHECKSIG",
            ]
        ),
        SIGHASH_ALL | SIGHASH_ANYONECANPAY,
    )
    # print(sig)

    # sign the second input
    sig2 = sk2.sign_input(
        tx,
        1,
        Script(
            [
                "OP_DUP",
                "OP_HASH160",
                from_addr2.to_hash160(),
                "OP_EQUALVERIFY",
                "OP_CHECKSIG",
            ]
        ),
        SIGHASH_SINGLE | SIGHASH_ANYONECANPAY,
    )
    # print(sig2)

    # get public key as hex
    pk = sk.get_public_key()
    pk = pk.to_hex()
    # print (pk)

    # get public key as hex
    pk2 = sk2.get_public_key()
    pk2 = pk2.to_hex()

    # set the scriptSig (unlocking script)
    txin.script_sig = Script([sig, pk])
    txin2.script_sig = Script([sig2, pk2])
    signed_tx = tx.serialize()

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + signed_tx)
    print("\nTxId:", tx.get_txid())

```

(continues on next page)

(continued from previous page)

```

if __name__ == "__main__":
    main()
```

2.6 SegWit

SegWit separates witness data from the transaction data used for the legacy transaction id. The library supports native SegWit v0 outputs and nested P2SH-SegWit outputs.

2.6.1 P2WPKH Addresses

```
from bitcoinutils.keys import PrivateKey

pub = PrivateKey().get_public_key()
address = pub.get_segwit_address()
script_pubkey = address.to_script_pub_key()
```

2.6.2 Spending P2WPKH

For P2WPKH, the signing script code is the P2PKH template using the public key hash.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
from bitcoinutils.keys import P2pkhAddress, PrivateKey
from bitcoinutils.script import Script


def main():
    # always remember to setup the network
    setup("testnet")

    # the key that corresponds to the P2WPKH address
    priv = PrivateKey("cVdte9ei2xsVjmZSPtyucG43YZgNkmKTqhwiUA8M4Fc3LdPJxPmZ")

    pub = priv.get_public_key()

    fromAddress = pub.get_segwit_address()
    print(fromAddress.to_string())

```

(continues on next page)

(continued from previous page)

```
    # amount is needed to sign the segwit input
    fromAddressAmount = to_satoshis(0.01)

    # UTXO of fromAddress
    txid = "13d2d30eca974e8fa5da11b9608fa36905a22215e8df895e767fc903889367ff"
    vout = 0

    toAddress = P2pkhAddress("mrrKUpJnAjvQntPgz2Z4kkyr1gbtHmQv28")

    # create transaction input from tx id of UTXO
    txin = TxInput(txid, vout)

    # in segwit the signature message should commit to the script code
    # that corresponds to the segwit transaction output
    # the script code (template) required for signing for p2wpkh is the
    # same as p2pkh
    script_code = Script(
        ["OP_DUP", "OP_HASH160", pub.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
    )

    # create transaction output
    txOut = TxOutput(to_satoshis(0.009), toAddress.to_script_pub_key())

    # create transaction without change output - if at least a single input is
    # segwit we need to set has_segwit=True
    tx = Transaction([txin], [txOut], has_segwit=True)

    print("\nRaw transaction:\n" + tx.serialize())

    sig = priv.sign_segwit_input(tx, 0, script_code, fromAddressAmount)

    # note that TxWitnessInput gets a list of witness items (not script opcodes)
    tx.witnesses.append(TxWitnessInput([sig, pub.to_hex()]))

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + tx.serialize())
    print("\nTxId:", tx.get_txid())


if __name__ == "__main__":
    main()
```

2.6.3 Nested P2SH-P2WPKH

Nested SegWit spends use a P2SH scriptSig containing the redeem script and a witness stack containing the signature and public key.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
```

(continues on next page)

(continued from previous page)

```
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.keys import PrivateKey, P2pkhAddress
from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
from bitcoinutils.script import Script


def main():
    # always remember to setup the network
    setup("testnet")

    # the key that corresponds to the P2WPKH address
    priv = PrivateKey("cNho8fw3bPfLKT4jPzpANTsxTsP8aTdVBD6cXksBEXt4KhBN7uVk")
    pub = priv.get_public_key()

    # the p2sh script and the corresponding address
    redeem_script = pub.get_segwit_address().to_script_pub_key()

    # the UTXO of the P2SH-P2WPKH that we are trying to spend
    inp = TxInput("95c5cac558a8b47436a3306ba300c8d7af4cd1d1523d35da3874153c66d99b09", 0)

    # exact amount of UTXO we try to spent
    amount = 0.0014

    # the address to send funds to
    to_addr = P2pkhAddress("mvBGdiYC8jLumpJ142ghePYuY8kecQgeqS")

    # the output sending 0.001 -- 0.0004 goes to miners as fee -- no change
    out = TxOutput(to_satoshis(0.001), to_addr.to_script_pub_key())

    # create a tx with at least one segwit input
    tx = Transaction([inp], [out], has_segwit=True)

    # script code is the script that is evaluated for a witness program type;
    # each witness program type has a specific template for the script code;
    # the script code that corresponds to P2WPKH is the same as P2PKH
    script_code = pub.get_address().to_script_pub_key()

    # calculate signature using the appropriate script code
    # remember to include the original amount of the UTXO
    sig = priv.sign_segwit_input(tx, 0, script_code, to_satoshis(amount))

    # script_sig is the redeem script passed as a single element
    inp.script_sig = Script([redeem_script.to_hex()])

    # finally, the unlocking script is added as a witness
```

(continues on next page)

(continued from previous page)

```
    # note that TxWitnessInput gets a list of witness items (not script opcodes)
    tx.witnesses.append(TxWitnessInput([sig, pub.to_hex()]))

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + tx.serialize())


if __name__ == "__main__":
    main()
```

2.6.4 P2WSH

P2WSH spends use the witness script as the signing script and include it as the last witness stack item.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
from bitcoinutils.keys import PrivateKey, P2wshAddress, P2wpkhAddress
from bitcoinutils.script import Script


def main():
    # always remember to setup the network
    setup("testnet")

    priv1 = PrivateKey("cN1XE3ESGgdvr4fWsB7L3BcqXncUauF8Fo8zzv4Sm6WrkiGrsxrG")
    priv2 = PrivateKey("cR8AkcbL2pgBswrHp28AftEznHPPLA86HiTog8MpNCibxwrsUcZ4")

    p2wsh_witness_script = Script(
        [
            "OP_1",
            priv1.get_public_key().to_hex(),
            priv2.get_public_key().to_hex(),
            "OP_2",
            "OP_CHECKMULTISIG",
        ]
    )

    fromAddress = P2wshAddress.from_script(p2wsh_witness_script)

    toAddress = P2wpkhAddress.from_address("tb1qtstf97nhk2gycz7vl37esddjpxwt3ut30qp5pn")
```

(continues on next page)

(continued from previous page)

```
    # set values
    txid = "2042195c40a92353f2ffe30cd0df8d177698560e81807e8bf9174a9c0e98e6c2"
    vout = 0
    amount = to_satoshis(0.01)

    # create transaction input from tx id of UTXO
    txin = TxInput(txid, vout)

    txOut1 = TxOutput(to_satoshis(0.0001), toAddress.to_script_pub_key())
    txOut2 = TxOutput(to_satoshis(0.0098), fromAddress.to_script_pub_key())

    tx = Transaction([txin], [txOut1, txOut2], has_segwit=True)

    sig1 = priv1.sign_segwit_input(tx, 0, p2wsh_witness_script, amount)

    # note that TxWitnessInput gets a list of witness items (not script opcodes)
    tx.witnesses.append(TxWitnessInput(["", sig1, p2wsh_witness_script.to_hex()]))

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + tx.serialize())
    print("\nTxId:", tx.get_txid())


if __name__ == "__main__":
    main()
```

2.7 Taproot

Taproot support includes P2TR addresses, key-path signing, script-path signing, Taproot tweaks, and control blocks.

2.7.1 Addresses and Tweaks

`PublicKey.get_taproot_address()` returns a `P2trAddress`. If scripts are passed, the internal key is tweaked with the script tree commitment.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.keys import P2trAddress, PrivateKey


```

(continues on next page)

(continued from previous page)

```
def main():
    # always remember to setup the network
    setup("testnet")

    # could also instantiate from existing WIF key
    priv = PrivateKey.from_wif("cRPxBiKrJsH94FLugmiL4xnezMyoFqGcf4kdgNXGuypNERhMK6AT")

    # compressed is the default
    print("\nPrivate key WIF:", priv.to_wif())

    # get the public key
    pub = priv.get_public_key()

    # public keys
    print("Public key as usual:", pub.to_hex())
    print("Taproot tweaked public key:", pub.to_taproot_hex())

    # get address from public key
    address = pub.get_taproot_address()

    # print the address and hash - default is compressed address
    print("Native Address:", address.to_string())
    taproot_pk = address.to_witness_program()
    print("Taproot witness program:", taproot_pk)
    print("Segwit Version:", address.get_type())

    # test to_string
    addr2 = P2trAddress.from_witness_program(taproot_pk)
    print("Created P2trAddress from public key and calculate address:")
    print("Native Address:", addr2.to_string())

    assert (
        address.to_string()
        == "tb1pdr8q4tuqqeglxxhkxl3trxt0dy5jrnaqvg0ddwu7plraxvntp8dqv8kvyq"
    )
    assert (
        addr2.to_string()
        == "tb1pdr8q4tuqqeglxxhkxl3trxt0dy5jrnaqvg0ddwu7plraxvntp8dqv8kvyq"
    )


if __name__ == "__main__":
    main()
```

2.7.2 Key-Path Spending

For a key-path spend, call `PrivateKey.sign_taproot_input` with all input scriptPubKeys and amounts.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
```

(continues on next page)

(continued from previous page)

```
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
from bitcoinutils.keys import P2pkhAddress, PrivateKey


def main():
    # always remember to setup the network
    setup("testnet")

    # the key that corresponds to the P2WPKH address
    priv = PrivateKey("cV3R88re3AZSBnWhBBNdiCKTfwpMKkYYjdiR13HQzsU7zoRNX7JL")

    pub = priv.get_public_key()

    fromAddress = pub.get_taproot_address()
    print(fromAddress.to_string())

    # UTXO of fromAddress
    txid = "7b6412a0eed56338731e83c606f13ebb7a3756b3e4e1dbbe43a7db8d09106e56"
    vout = 1

    # all amounts are needed to sign a taproot input
    # (depending on sighash)
    first_amount = to_satoshis(0.00005)
    amounts = [first_amount]

    # all scriptPubKeys are needed to sign a taproot input
    # (depending on sighash) but always of the spend input
    first_script_pubkey = fromAddress.to_script_pub_key()

    # alternatively:
    # first_script_pubkey = Script(['OP_1', pub.to_taproot_hex()])

    utxos_script_pubkeys = [first_script_pubkey]

    toAddress = P2pkhAddress("mtVHHCqCECGwiMbMoZe8ayhJHuTdDbYWdJ")

    # create transaction input from tx id of UTXO
    txin = TxInput(txid, vout)

    # create transaction output
    txOut = TxOutput(to_satoshis(0.00004), toAddress.to_script_pub_key())

    # create transaction without change output - if at least a single input is
```

(continues on next page)

(continued from previous page)

```
    # segwit we need to set has_segwit=True
    tx = Transaction([txin], [txOut], has_segwit=True)

    print("\nRaw transaction:\n" + tx.serialize())

    print("\ntxid: " + tx.get_txid())
    print("\ntxwid: " + tx.get_wtxid())

    # sign taproot input
    # to create the digest message to sign in taproot we need to
    # pass all the utxos' scriptPubKeys and their amounts
    sig = priv.sign_taproot_input(tx, 0, utxos_script_pubkeys, amounts)
    # print(sig)

    tx.witnesses.append(TxWitnessInput([sig]))

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + tx.serialize())

    print("\nTxId:", tx.get_txid())
    print("\nTxwId:", tx.get_wtxid())

    print("\nSize:", tx.get_size())
    print("\nvSize:", tx.get_vsize())


if __name__ == "__main__":
    main()
```

2.7.3 Script-Path Spending

Script-path spends add the executed tapleaf script and a control block to the witness stack.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis, ControlBlock
from bitcoinutils.script import Script
from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
from bitcoinutils.keys import PrivateKey
from bitcoinutils.hdwallet import HDWallet


```

(continues on next page)

(continued from previous page)

```
def main():
    # always remember to setup the network
    setup("testnet")

    # Keys are hard-coded in the example for simplicity but it is very bad
    # practice. Normally you would acquire them from env variables, db, etc

    ########################
    # Construct the input #
    ########################

    # get an HDWallet wrapper object by extended private key and path
    xprivkey = (
        "tprv8ZgxMBicQKsPdQR9RuHpGGxSnNq8Jr3X4WnT6Nf2eq7FajuXyBep5KWYpYEixxx5XdTm1N"
        "tpe84f3cVcF7mZZ7mPkntaFXLGJD2tS7YJkWU"
    )
    path = "m/86'/1'/0'/0/7"
    hdw = HDWallet(xprivkey, path)
    internal_priv = hdw.get_private_key()
    print("From Private key:", internal_priv.to_wif())
    internal_pub = internal_priv.get_public_key()
    print("From Public key:", internal_pub.to_hex())

    # taproot script is a simple P2PK with the following keys
    privkey_tr_script = PrivateKey(
        "cQwzrJyTNWbEwhPEmQ3Qoo4jSfHdHEtdbL4kNBgHUKhirgzcQw7G"
    )
    pubkey_tr_script = privkey_tr_script.get_public_key()
    tr_script_p2pk = Script([pubkey_tr_script.to_x_only_hex(), "OP_CHECKSIG"])

    # taproot script path address
    from_address = internal_pub.get_taproot_address([[tr_script_p2pk]])
    print("From Taproot script address", from_address.to_string())

    # UTXO of from_address from pubkey 02
    txid = "348f577ae2509b3b73ebd810c3cdcb18045ef62b43378aed283b3259afe493b1"
    vout = 0

    # create transaction input from tx id of UTXO
    tx_in = TxInput(txid, vout)

    # all amounts are needed to sign a taproot input
    # (depending on sighash)
    amount = to_satoshis(0.00009)
    amounts = [amount]

    # all scriptPubKeys (in hex) are needed to sign a taproot input
    # (depending on sighash but always of the spend input)
    scriptPubkey = from_address.to_script_pub_key()
    utxos_scriptPubkeys = [scriptPubkey]

    ########################
```

(continues on next page)

(continued from previous page)

```
    # Construct the output #
    ########################

    hdw.from_path("m/86'/1'/0'/0/5")
    to_priv = hdw.get_private_key()
    print("To Private key:", to_priv.to_wif())

    to_pub = to_priv.get_public_key()
    print("To Public key:", to_pub.to_hex())

    # taproot key path address
    to_address = to_pub.get_taproot_address()
    print("To Taproot address:", to_address.to_string())

    # create transaction output
    tx_out = TxOutput(to_satoshis(0.000085), to_address.to_script_pub_key())

    # create transaction without change output - if at least a single input is
    # segwit we need to set has_segwit=True
    tx = Transaction([tx_in], [tx_out], has_segwit=True)

    print("\nRaw transaction:\n" + tx.serialize())

    print("\ntxid: " + tx.get_txid())
    print("\ntxwid: " + tx.get_wtxid())

    # sign taproot input
    # to create the digest message to sign in taproot we need to
    # pass all the utxos' scriptPubKeys, their amounts and taproot script
    # we sign with the private key corresponding to the script - no keys
    # tweaking required
    sig = privkey_tr_script.sign_taproot_input(
        tx,
        0,
        utxos_scriptPubkeys,
        amounts,
        script_path=True,
        tapleaf_script=tr_script_p2pk,
        tweak=False,
    )

    control_block = ControlBlock(internal_pub, scripts=[tr_script_p2pk], index=0, is_
↪odd=to_address.is_odd())

    tx.witnesses.append(
        TxWitnessInput([sig, tr_script_p2pk.to_hex(), control_block.to_hex()])
    )

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + tx.serialize())

    print("\nTxId:", tx.get_txid())
```

(continues on next page)

(continued from previous page)

```
    print("\nTxwId:", tx.get_wtxid())

    print("\nSize:", tx.get_size())
    print("\nvSize:", tx.get_vsize())


if __name__ == "__main__":
    main()
```

2.7.4 Multiple Script Paths

The examples directory contains two-, three-, and four-leaf Taproot trees.

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

from bitcoinutils.setup import setup
from bitcoinutils.utils import to_satoshis
from bitcoinutils.script import Script
from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
from bitcoinutils.keys import PrivateKey
from bitcoinutils.hdwallet import HDWallet


def main():
    # always remember to setup the network
    setup("testnet")

    ########################
    # Construct the input #
    ########################

    # get an HDWallet wrapper object by extended private key and path
    xprivkey = (
        "tprv8ZgxMBicQKsPdQR9RuHpGGxSnNq8Jr3X4WnT6Nf2eq7FajuXyBep5KWYpYEixxx5XdTm1N"
        "tpe84f3cVcF7mZZ7mPkntaFXLGJD2tS7YJkWU"
    )
    path = "m/86'/1'/0'/0/5"
    hdw = HDWallet(xprivkey, path)
    internal_priv = hdw.get_private_key()
    print("From Private key:", internal_priv.to_wif())

    internal_pub = internal_priv.get_public_key()
    print("From Public key:", internal_pub.to_hex())
```

(continues on next page)

(continued from previous page)

```
    from_address = internal_pub.get_taproot_address()
    print("From Taproot address:", from_address.to_string())

    # UTXO of fromAddress
    txid = "29afd65f1aeeab4e4d655b148776fe0097acc617492b0c3f3950b6a95be20f39"
    vout = 0

    # create transaction input from tx id of UTXO
    tx_in = TxInput(txid, vout)

    # all amounts are needed to sign a taproot input
    # (depending on sighash)
    amount = to_satoshis(0.00004)
    amounts = [amount]

    # all scriptPubKeys (in hex) are needed to sign a taproot input
    # (depending on sighash but always of the spend input)
    scriptPubkey = from_address.to_script_pub_key()
    utxos_scriptPubkeys = [scriptPubkey]

    ########################
    # Construct the output #
    ########################

    hdw.from_path("m/86'/1'/0'/0/7")
    to_priv = hdw.get_private_key()
    print("Send to Private key", to_priv.to_wif())
    to_pub = to_priv.get_public_key()

    # taproot script A is a simple P2PK with the following keys
    privkey_tr_script_A = PrivateKey(
        "cSW2kQbqC9zkqagw8oTYKFTozKuZ214zd6CMTDs4V32cMfH3dgKa"
    )
    pubkey_tr_script_A = privkey_tr_script_A.get_public_key()
    tr_script_p2pk_A = Script([pubkey_tr_script_A.to_x_only_hex(), "OP_CHECKSIG"])

    # taproot script B is a simple P2PK with the following keys
    privkey_tr_script_B = PrivateKey(
        "cSv48xapaqy7fPs8VvoSnxNBNA2jpjcuURRqUENu3WVq6Eh4U3JU"
    )
    pubkey_tr_script_B = privkey_tr_script_B.get_public_key()
    tr_script_p2pk_B = Script([pubkey_tr_script_B.to_x_only_hex(), "OP_CHECKSIG"])

    # taproot script C is a simple P2PK with the following keys
    privkey_tr_script_C = PrivateKey(
        "cRkZPNnn3jdr64o3PDxNHG68eowDfuCdcyL6nVL4n3czvunuvryC"
    )
    pubkey_tr_script_C = privkey_tr_script_C.get_public_key()
    tr_script_p2pk_C = Script([pubkey_tr_script_C.to_x_only_hex(), "OP_CHECKSIG"])

    # tapleafs in order
```

(continues on next page)

(continued from previous page)

```
    #                   TB_ABC
    #                   /    \
    #                  /      \
    #               TB_AB      \
    #               / \         \
    #              /   \         \
    #            TL_A TL_B      TL_C
    all_leafs = [[tr_script_p2pk_A, tr_script_p2pk_B], tr_script_p2pk_C]

    # taproot script path address
    to_address = to_pub.get_taproot_address(all_leafs)
    print("To Taproot script address", to_address.to_string())

    # create transaction output
    tx_out = TxOutput(to_satoshis(0.000035), to_address.to_script_pub_key())

    # create transaction without change output - if at least a single input is
    # segwit we need to set has_segwit=True
    tx = Transaction([tx_in], [tx_out], has_segwit=True)

    print("\nRaw transaction:\n" + tx.serialize())

    print("\ntxid: " + tx.get_txid())
    print("\ntxwid: " + tx.get_wtxid())

    # sign taproot input
    # to create the digest message to sign in taproot we need to
    # pass all the utxos' scriptPubKeys and their amounts
    sig = internal_priv.sign_taproot_input(tx, 0, utxos_scriptPubkeys, amounts)

    tx.witnesses.append(TxWitnessInput([sig]))

    # print raw signed transaction ready to be broadcasted
    print("\nRaw signed transaction:\n" + tx.serialize())

    print("\nTxId:", tx.get_txid())
    print("\nTxwId:", tx.get_wtxid())

    print("\nSize:", tx.get_size())
    print("\nvSize:", tx.get_vsize())


if __name__ == "__main__":
    main()
```

2.8 HD Wallets

`bitcoinutils.hdwallet.HDWallet` implements the small BIP-32/BIP-39 subset used by the examples: derive private keys from a mnemonic or an extended private key and return them as `PrivateKey` objects.

2.8.1 From a Mnemonic

```
from bitcoinutils.hdwallet import HDWallet

mnemonic = "addict weather world sense idle purity rich wagon ankle fall cheese spatial"
wallet = HDWallet.from_mnemonic(mnemonic)
wallet.from_path("m/84'/1'/0'/0/0")

private_key = wallet.get_private_key()
print(private_key.get_public_key().get_segwit_address().to_string())
```

2.8.2 From an Extended Private Key

```
wallet = HDWallet.from_xprivate_key(tprv, "m/86'/1'/0'/0/0")
print(wallet.get_private_key().get_public_key().get_taproot_address().to_string())
```

2.8.3 Paths

Only absolute paths starting with `m/` are supported. Hardened components may use `'`, `h`, or `H`.

Common testnet examples:

- `m/44'/1'/0'/0/0` for legacy P2PKH intent.
- `m/84'/1'/0'/0/0` for native SegWit P2WPKH intent.
- `m/86'/1'/0'/0/0` for Taproot P2TR intent.

2.8.4 Example

```
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.

"""Detailed HD wallet examples.

This example demonstrates the minimal HD functionality provided by
bitcoinutils:

* create a wallet from a BIP-39 mnemonic
* derive private keys from different BIP-style paths
* derive legacy, segwit v0, and taproot addresses from the same key
* switch paths on the same HDWallet instance
* create a wallet from an extended private key

The mnemonic and extended private keys below are for testnet examples only.
Do not use them with real funds.
```

(continues on next page)

(continued from previous page)

```
"""

from bitcoinutils.setup import setup
from bitcoinutils.hdwallet import HDWallet


def print_key_details(title, hdw):
    """Print the currently selected key and common address types."""

    privkey = hdw.get_private_key()
    pubkey = privkey.get_public_key()

    print(f"\n{title}")
    print("-" * len(title))
    print("WIF:", privkey.to_wif())
    print("Public key:", pubkey.to_hex())
    print("Legacy P2PKH:", pubkey.get_address().to_string())
    print("Native Segwit P2WPKH:", pubkey.get_segwit_address().to_string())
    print("Taproot P2TR:", pubkey.get_taproot_address().to_string())


def derive_and_print(hdw, path):
    """Derive an absolute path and print the resulting key/address details."""

    hdw.from_path(path)
    print_key_details(f"Path {path}", hdw)


def main():
    setup("testnet")

    mnemonic = (
        "addict weather world sense idle purity rich wagon "
        "ankle fall cheese spatial"
    )

    print("Mnemonic wallet")
    print("===============")
    print("Mnemonic:", mnemonic)

    hdw_from_mnemonic = HDWallet(mnemonic=mnemonic)

    # BIP-44: legacy P2PKH-style account path.
    derive_and_print(hdw_from_mnemonic, "m/44'/1'/0'/0/0")
    derive_and_print(hdw_from_mnemonic, "m/44'/1'/0'/0/1")

    # BIP-84: native segwit account path. The private/public key can still be
    # rendered as multiple address types; the path communicates wallet intent.
    derive_and_print(hdw_from_mnemonic, "m/84'/1'/0'/0/0")

    # BIP-86: taproot account path.
    derive_and_print(hdw_from_mnemonic, "m/86'/1'/0'/0/0")
```

(continues on next page)

(continued from previous page)

```
    print("\nSwitching paths")
    print("===============")
    hdw_from_mnemonic.from_path("m/44'/1'/0'/0/3")
    addr_a = hdw_from_mnemonic.get_private_key().get_public_key().get_address()
    print("m/44'/1'/0'/0/3 legacy address:", addr_a.to_string())

    hdw_from_mnemonic.from_path("m/44'/1'/0'/0/4")
    addr_b = hdw_from_mnemonic.get_private_key().get_public_key().get_address()
    print("m/44'/1'/0'/0/4 legacy address:", addr_b.to_string())

    xprivkey = (
        "tprv8ZgxMBicQKsPdQR9RuHpGGxSnNq8Jr3X4WnT6Nf2eq7FajuXyBep5KWYpYEixxx5XdTm1N"
        "tpe84f3cVcF7mZZ7mPkntaFXLGJD2tS7YJkWU"
    )

    print("\nExtended private key wallet")
    print("===========================")
    print("Extended private key:", xprivkey)

    hdw_from_xpriv = HDWallet.from_xprivate_key(xprivkey, "m/86'/1'/0'/0/1")
    print_key_details("Path m/86'/1'/0'/0/1", hdw_from_xpriv)

    hdw_from_xpriv.from_path("m/86'/1'/0'/0/5")
    print_key_details("Path m/86'/1'/0'/0/5", hdw_from_xpriv)


if __name__ == "__main__":
    main()
```

2.9 Descriptors

`bitcoinutils.descriptors` implements a small educational subset of Bitcoin Core output descriptors. It is intended to show how descriptor text maps to scripts and addresses while still leaving transaction construction, fees, change, and UTXO selection explicit.

2.9.1 Supported Forms

The first version supports fixed public keys and addresses:

- `pk(KEY)`
- `pkh(KEY)`
- `wpkh(KEY)`
- `sh(wpkh(KEY))`
- `multi(k,KEY...)` and `sortedmulti(k,KEY...)`
- `sh(multi(...))` and `sh(sortedmulti(...))`
- `wsh(multi(...))` and `wsh(sortedmulti(...))`
- `sh(wsh(multi(...)))` and `sh(wsh(sortedmulti(...)))`

- `tr(KEY)` for key-path Taproot
- `addr(ADDRESS)`
- `raw(HEX)`

Xpubs, derivation paths, wildcards, key-origin metadata, Miniscript, Taproot script trees, `combo`, MuSig2, and private keys are intentionally not supported in this version.

2.9.2 Example

```
from bitcoinutils.descriptors import parse_descriptor, add_descriptor_checksum

desc = parse_descriptor(
    "wpkh(0279be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798)"
)

print(desc.to_script_pub_key().to_hex())
print(desc.to_address().to_string())
print(add_descriptor_checksum(desc.to_string()))
```

2.9.3 Nested SegWit

```
native = parse_descriptor(
↪"wpkh(0279be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798)")
nested = parse_descriptor(
↪"sh(wpkh(0279be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798))")

print(native.to_script_pub_key().to_hex())
print(nested.to_script_pub_key().to_hex())
```

2.9.4 Checksums

Descriptors may include Bitcoin Core compatible checksums:

```
desc = parse_descriptor("addr(mkmZxiEcEd8ZqjQWVZuC6so5dFMKEFpN2j)#02wpgw69", require_
↪checksum=True)
print(desc.validate_checksum())
```

2.10 PSBT

The `bitcoinutils.psbt` module implements BIP-174 PSBT v0 workflows.

2.10.1 Roles

The `PSBT` class can act as:

- Creator: wrap an unsigned `Transaction`.
- Updater: attach UTXO and script metadata with `update_input`.
- Signer: call `sign_input` with a `PrivateKey`.
- Combiner: merge compatible PSBTs with `combine`.
- Finalizer: call `finalize` or `finalize_input`.

- Extractor: call `extract_transaction`.

2.10.2 Minimal Flow

```
psbt = PSBT(tx)
psbt.update_input(0, non_witness_utxo=previous_tx, redeem_script=redeem_script)
psbt.sign_input(0, private_key)
psbt.finalize()
final_tx = psbt.extract_transaction()
```

2.10.3 2-of-3 Multisig Walkthrough

The examples include a complete 2-of-3 P2SH multisig PSBT workflow split across creator and signer scripts.

```
# PSBT 2-of-3 Multisig Signing Guide

This document describes how to use BIP-174 PSBTs to coordinate a **2-of-3 P2SH multisig** spend across two independent signers. This is one of the most common real-world PSBT use cases: a transaction that requires two out of three keyholders to approve before it can be broadcast.

We simulate the workflow using three scripts (in `examples/`) that each execute one or more BIP-174 roles. Both signing scripts run on the same machine but use different private keys, just as they would if two keyholders were signing on separate devices or in separate locations.

```bash
python examples/psbt_2of3_create.py    # Step 1: Create unsigned PSBT
python examples/psbt_2of3_sign1.py     # Step 2: Signer 1 adds partial signature
python examples/psbt_2of3_sign2.py     # Step 3: Signer 2 signs, combines, finalizes, extracts
```

Background

What is 2-of-3 multisig?

A 2-of-3 multisig address is controlled by three public keys, and any two of the three corresponding private keys are required to spend funds from it. The locking script (redeemScript) looks like:

```
OP_2 <pubkey_1> <pubkey_2> <pubkey_3> OP_3 OP_CHECKMULTISIG
```

The address itself is the hash of this redeemScript, wrapped in P2SH format (`3...` on mainnet, `2...` on testnet).

Why PSBT?

Without PSBT, coordinating multiple signers requires passing around raw transaction bytes and manually managing partial signatures. PSBT solves this by defining a standard container that carries:
```

(continues on next page)

(continued from previous page)

```
28
29  - The unsigned transaction
30  - UTXO data needed for signing
31  - Script metadata (redeemScript, witnessScript)
32  - Partial signatures from each signer
33
34  Each participant adds their piece and passes the PSBT along. No signer needs to␣
    ↪understand what the others are doing; they just sign and pass it on.
35
36  ### BIP-174 roles in this workflow
37
38  BIP-174 defines six roles. In our 2-of-3 scenario, they map to three scripts:
39
40  | Role | Responsibility | Script |
41  |---|---|---|
42  | **Creator** | Builds the unsigned transaction and wraps it in a PSBT | `psbt_2of3_
    ↪create.py` |
43  | **Updater** | Adds the UTXO info, redeemScript, and sighash type | `psbt_2of3_create.
    ↪py` |
44  | **Signer 1** | Adds a partial signature using private key A | `psbt_2of3_sign1.py` |
45  | **Signer 2** | Adds a partial signature using private key B | `psbt_2of3_sign2.py` |
46  | **Combiner** | Merges the two signed PSBTs into one | `psbt_2of3_sign2.py` |
47  | **Finalizer** | Constructs the final scriptSig from partial signatures | `psbt_2of3_
    ↪sign2.py` |
48  | **Extractor** | Pulls out the fully signed transaction for broadcast | `psbt_2of3_
    ↪sign2.py` |
49
50  Note: the Creator and Updater roles are combined in the first script, and the Combiner/
    ↪Finalizer/Extractor are combined in the second signer's script. This is a common␣
    ↪pattern - in practice the last signer usually finalizes and extracts.
51
52  ## Setup
53
54  ### Shared knowledge (all participants know this)
55
56  Before the workflow starts, all three keyholders have agreed on:
57
58  1. **The three public keys** that form the multisig (and their order in the␣
    ↪redeemScript).
59  2. **The redeemScript** itself, derived from those three public keys.
60  3. **The P2SH address** derived from the redeemScript - this is where funds are locked.
61
62  ### Private key distribution
63
64  - **Key A** - held by Signer 1 only
65  - **Key B** - held by Signer 2 only
66  - **Key C** - held by a third party (cold storage, escrow, etc.) - not used in this spend
67
68  Any two of the three can sign. In this example, Signers 1 and 2 cooperate.
69
70  ## Workflow
71
```

(continues on next page)

(continued from previous page)

The PSBT flows through the scripts as a **base64-encoded string**. In production this string would be passed via file, QR code, NFC, or a coordination server. Here we write it to `.psbt` files on disk.

```
              psbt_2of3_create.py
              (Creator + Updater)
                     |
                     v
              unsigned.psbt          <-- base64 PSBT with UTXO + redeemScript
                     |
         +-----------+-----------+
         |                       |
         v                       v
  psbt_2of3_sign1.py   (copy of unsigned.psbt)
  (Signer 1)                     |
         |                       |
         v                       |
  signer1.psbt                   |
  (has sig from Key A)           |
         |                       |
         +-----------+-----------+
                     |
                     v
            psbt_2of3_sign2.py
            (Signer 2 + Combiner +
             Finalizer + Extractor)
                     |
                     v
             final_tx.hex          <-- raw signed transaction, ready to broadcast
```

Step 1: Create the PSBT (`psbt_2of3_create.py`)

This script acts as both **Creator** and **Updater**.

Creator responsibilities:
- Build a `Transaction` with the desired inputs (UTXOs locked in the multisig address) and outputs (where the funds are going).
- Wrap it in a `PSBT` object. The constructor automatically strips scriptSigs and segwit data, producing a clean unsigned transaction.

Updater responsibilities:
- Attach the **non-witness UTXO** (the full previous transaction) for each input. This lets signers verify the amount being spent.
- Attach the **redeemScript** for each input. Without it, signers cannot determine what they are signing for.

```python
# Pseudocode for psbt_2of3_create.py

setup("testnet")
```

(continues on next page)

(continued from previous page)

```
# 1. Define the 3 public keys and build the redeemScript
pk1 = PublicKey(...)
pk2 = PublicKey(...)
pk3 = PublicKey(...)

redeem_script = Script([
    "OP_2",
    pk1.to_hex(), pk2.to_hex(), pk3.to_hex(),
    "OP_3", "OP_CHECKMULTISIG"
])

# 2. Create the unsigned transaction (Creator)
txin = TxInput(prev_txid, vout_index)
txout = TxOutput(amount, destination_script_pubkey)
tx = Transaction([txin], [txout])

# 3. Wrap in PSBT
psbt = PSBT(tx)

# 4. Update with UTXO and redeemScript (Updater)
psbt.update_input(0,
    non_witness_utxo=prev_tx,       # full previous transaction
    redeem_script=redeem_script      # so signers know the multisig structure
)

# 5. Export
with open("unsigned.psbt", "w") as f:
    f.write(psbt.to_base64())
```

What the PSBT contains at this point:
- Global: the unsigned transaction
- Input 0: `non_witness_utxo` (full prev tx) + `redeem_script` (the 2-of-3 script)
- Input 0: no signatures yet

Step 2: Signer 1 signs (`psbt_2of3_sign1.py`)

This script acts as **Signer 1** only.

Signer responsibilities (per BIP-174):
- Parse the PSBT.
- Verify the UTXO data is consistent (txid of `non_witness_utxo` matches the input's referenced txid).
- Produce a partial signature using their private key.
- Add the signature to the PSBT as a `partial_sig` entry.
- Export the updated PSBT.

```python
# Pseudocode for psbt_2of3_sign1.py

setup("testnet")
```

(continues on next page)

(continued from previous page)

```
# 1. Load the unsigned PSBT
with open("unsigned.psbt") as f:
    psbt = PSBT.from_base64(f.read().strip())

# 2. Load Signer 1's private key
sk1 = PrivateKey("cXXXX...")  # Key A (WIF format)

# 3. Sign input 0
psbt.sign_input(0, sk1)

# 4. Export - DO NOT finalize (need Signer 2's signature too)
with open("signer1.psbt", "w") as f:
    f.write(psbt.to_base64())
```

What the PSBT contains at this point:
- Everything from before, plus:
- Input 0: `partial_sigs` now has one entry: `{pubkey_A: signature_A}`

Important: Signer 1 must NOT finalize the PSBT. Finalization locks in the scriptSig and clears partial signatures. Since we need a second signature, the PSBT must remain in its partially-signed state.

Step 3: Signer 2 signs, combines, finalizes, and extracts (`psbt_2of3_sign2.py`)

This script acts as **Signer 2**, **Combiner**, **Finalizer**, and **Extractor**.

```python
# Pseudocode for psbt_2of3_sign2.py

setup("testnet")

# 1. Load both PSBTs
with open("unsigned.psbt") as f:
    psbt_unsigned = PSBT.from_base64(f.read().strip())

with open("signer1.psbt") as f:
    psbt_signed1 = PSBT.from_base64(f.read().strip())

# 2. Sign the unsigned copy with Signer 2's key (Signer)
sk2 = PrivateKey("cYYYY...")  # Key B (WIF format)
psbt_unsigned.sign_input(0, sk2)
# psbt_unsigned now has: partial_sigs = {pubkey_B: signature_B}

# 3. Combine the two PSBTs (Combiner)
combined = psbt_signed1.combine(psbt_unsigned)
# combined now has: partial_sigs = {pubkey_A: sig_A, pubkey_B: sig_B}

# 4. Finalize (Finalizer)
combined.finalize()
# This constructs the final scriptSig:
```

(continues on next page)

(continued from previous page)

```
#   OP_0 <sig_A> <sig_B> <redeemScript>
# and clears partial_sigs, redeem_script, etc.

# 5. Extract the signed transaction (Extractor)
final_tx = combined.extract_transaction()
tx_hex = final_tx.serialize()

with open("final_tx.hex", "w") as f:
    f.write(tx_hex)

print(f"Transaction ready to broadcast: {tx_hex}")
```

What happens during finalization:

The `finalize()` method detects that the input is a P2SH multisig (by inspecting the redeemScript). It:
1. Collects the partial signatures from `partial_sigs`.
2. Orders them to match the pubkey order in the redeemScript (Bitcoin's `OP_CHECKMULTISIG` requires this).
3. Builds the scriptSig: `OP_0 <sig_1> <sig_2> <serialized_redeemScript>`.
4. Clears all non-final fields (`partial_sigs`, `redeem_script`, `bip32_derivs`, `sighash_type`).

The `extract_transaction()` method takes the finalized scriptSig and places it into a standard Bitcoin transaction that can be serialized and broadcast.

Alternative flow: sequential signing

Instead of combining two separately-signed PSBTs, Signer 2 can sign the PSBT that Signer 1 already signed:

```
unsigned.psbt  -->  Signer 1 signs  -->  signer1.psbt  -->  Signer 2 signs  --> finalize + extract
```

In this flow, Signer 2's script is simpler:

```python
# Load Signer 1's output directly
with open("signer1.psbt") as f:
    psbt = PSBT.from_base64(f.read().strip())

# Sign on top of it (adds a second partial_sig)
sk2 = PrivateKey("cYYYY...")
psbt.sign_input(0, sk2)
# psbt now has: partial_sigs = {pubkey_A: sig_A, pubkey_B: sig_B}

# Finalize and extract (no combine needed)
psbt.finalize()
final_tx = psbt.extract_transaction()
```

(continues on next page)

(continued from previous page)

```
```

This is simpler but requires Signer 2 to wait for Signer 1. The combine-based flow allows both signers to work in parallel.

File summary

File	BIP-174 Roles	Input	Output
`psbt_2of3_create.py`	Creator + Updater	UTXO details, 3 public keys	`unsigned.psbt`
`psbt_2of3_sign1.py`	Signer	`unsigned.psbt`, Key A	`signer1.psbt`
`psbt_2of3_sign2.py`	Signer + Combiner + Finalizer + Extractor	`unsigned.psbt`, `signer1.psbt`, Key B	`final_tx.hex`

Key points

- **Never finalize early.** Only finalize after all required signatures (2 of 3) are present. Finalizing with fewer signatures produces an invalid transaction.
- **Public key order matters.** `OP_CHECKMULTISIG` expects signatures in the same order as the public keys in the redeemScript. The PSBT finalizer handles this automatically by matching pubkeys from `partial_sigs` against the redeemScript.
- **The PSBT is not confidential.** Anyone who sees the PSBT can see the transaction details and public keys. However, they cannot forge signatures without the private keys.
- **The redeemScript must be attached.** Without it in the PSBT, signers cannot determine what type of script they are signing for, and the finalizer cannot construct the correct scriptSig.
- **UTXO data is mandatory.** The `non_witness_utxo` (full previous transaction) lets each signer independently verify the amount being spent, preventing fee-manipulation attacks.

Extending to P2SH-P2WSH (segwit multisig)

For a segwit-wrapped multisig (P2SH-P2WSH), the flow is identical except:

1. The `update_input` call also includes a `witness_script` (the raw multisig script) and a `witness_utxo` (the specific output being spent).
2. The `redeem_script` becomes `OP_0 <SHA256(witness_script)>` - a P2WSH wrapper.
3. Finalization produces both a scriptSig (pushing the redeemScript) and a witness stack (`OP_0 <sig_1> <sig_2> <witness_script>`).

The PSBT library handles these differences transparently. The signing scripts are nearly identical; only the `update_input` call changes.

2.11 Blocks

The block module parses Bitcoin block headers and full raw blocks.

2.11.1 Block Headers

`BlockHeader.from_raw` accepts 80-byte header data as hex or bytes.

```
from bitcoinutils.block import BlockHeader

header = BlockHeader.from_raw(raw_header_hex)
print(header.get_block_hash())
print(header.format_timestamp())
print(header.get_target_hex())
```

2.11.2 Blocks

`Block.from_raw` parses magic bytes, size, header, transaction count, and all transactions.

```
from bitcoinutils.block import Block

block = Block.from_raw(raw_block_hex)
print(block.get_transactions_count())
print(block.get_coinbase_transaction())
```

2.11.3 Example

```
import os
import struct
import sys

# Dynamically add the root path to sys.path
root_path = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
if root_path not in sys.path:
    sys.path.append(root_path)

from bitcoinutils.block import Block
from bitcoinutils.constants import BLOCK_MAGIC_NUMBER

def main():
    if len(sys.argv) != 2:
        print("Usage: python script.py <path_to_blkNNNNN.dat>")
        return

    filename = sys.argv[1]

    try:
        with open(filename, 'rb') as file:
            block_height = 0
            while True:
                # Read 8 bytes: 4 for magic number and 4 for block size
                preamble = file.read(8)
                if len(preamble) < 8:
                    print("End of file reached.")
                    break

                magic, size = struct.unpack('<4sI', preamble)
```

(continues on next page)

(continued from previous page)

```
                magic_hex = magic.hex()

                if magic_hex not in BLOCK_MAGIC_NUMBER:
                    raise ValueError(f"Unknown or unsupported network magic number:
↪{magic_hex}")

                network = BLOCK_MAGIC_NUMBER[magic_hex]
                print(f"\n[{network}] Block #{block_height}")

                block_data = file.read(size)
                if len(block_data) < size:
                    print("Warning: Truncated block data.")
                    break

                raw_block = preamble + block_data
                block = Block.from_raw(raw_block)

                # Output block info
                print("Block Hash:", block.get_block_header().get_block_hash())
                print("Transactions:", block.get_transactions_count())

                block_height += 1

    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == '__main__':
    main()
```

2.12 Bitcoin Core RPC Proxy

`NodeProxy` is a small wrapper around Bitcoin Core JSON-RPC. It is useful for examples that need to query a local node or broadcast a raw transaction.

2.12.1 Basic Use

```
from bitcoinutils.proxy import NodeProxy

proxy = NodeProxy("bitcoinrpc", "password", host="127.0.0.1", port=18443)
block_count = proxy.getblockcount()
print(block_count)
```

2.12.2 Method Calls

RPC methods are exposed dynamically. Calling `proxy.getblockcount()` maps to the Bitcoin Core RPC method `getblockcount`.

2.12.3 Example

```python
# Copyright (C) 2018-2025 The python-bitcoin-utils developers
#
# This file is part of python-bitcoin-utils
#
# It is subject to the license terms in the LICENSE file found in the top-level
# directory of this distribution.
#
# No part of python-bitcoin-utils, including this file, may be copied,
# modified, propagated, or distributed except according to the terms contained
# in the LICENSE file.


from bitcoinutils.setup import setup
from bitcoinutils.proxy import NodeProxy


def main():
    # always remember to setup the network
    setup("regtest")

    # get a node proxy using default host and port
    proxy = NodeProxy("rpcuser", "rpcpw")

    # call the node's getblockcount JSON-RPC method
    count = proxy.getblockcount()

    # call the node's getblockhash JSON-RPC method
    block_hash = proxy.getblockhash(count)

    # call the node's getblock JSON-RPC method and print result
    block = proxy.getblock(block_hash)

    print("--- Block Information ---")
    print(f"Height: {block['height']}")
    print(f"Hash: {block['hash']}")
    print(f"Transactions: {len(block['tx'])}")
    print(f"Difficulty: {block['difficulty']}")

    print("\n--- Blockchain Information ---")
    binfo = proxy.getblockchaininfo()
    print(f"Chain: {binfo['chain']}")
    print(f"Blocks: {binfo['blocks']}")
    print(f"Size on disk: {binfo['size_on_disk']} bytes")

    print("\n--- Network Information ---")
    ninfo = proxy.getnetworkinfo()
    print(f"Version: {ninfo['version']}")
    print(f"Subversion: {ninfo['subversion']}")
    print(f"Connections: {ninfo['connections']}")

    print("\n--- Mempool Information ---")
```

(continues on next page)

(continued from previous page)

```
    minfo = proxy.getmempoolinfo()
    print(f"Size: {minfo['size']} transactions")
    print(f"Bytes: {minfo['bytes']} bytes")

    print("\n--- Wallet Information ---")
    try:
        # These commands require a loaded wallet
        balance = proxy.getbalance()
        print(f"Balance: {balance} BTC")

        new_addr = proxy.getnewaddress()
        print(f"New address: {new_addr}")
    except Exception as e:
        print(f"Wallet commands failed (no wallet loaded?): {e}")


if __name__ == "__main__":
    main()
```

2.13 Setup API

bitcoinutils.setup.**setup**(*network='testnet'*)

Parameters
network (*str*)

Return type
str

bitcoinutils.setup.**get_network**()

Return type
str

bitcoinutils.setup.**set_security_warnings**(*enabled*)

Enable or disable warnings for pure-Python private-key operations.

Parameters
enabled (*bool*)

Return type
None

bitcoinutils.setup.**get_security_warnings**()

Return whether pure-Python private-key operation warnings are enabled.

Return type
bool

bitcoinutils.setup.**should_warn_about_private_key_use**()

Return True the first time a private-key operation should warn.

Return type
bool

bitcoinutils.setup.**is_mainnet**()

Return type
bool

bitcoinutils.setup.**is_testnet**()

Return type
bool

bitcoinutils.setup.**is_testnet4**()

Return type
bool

bitcoinutils.setup.**is_signet**()

Return type
bool

bitcoinutils.setup.**is_regtest**()

Return type
bool

2.14 Keys and Addresses API

class bitcoinutils.keys.**PrivateKey**(*wif=None*, *secret_exponent=None*, *b=None*)

Bases: `object`

Represents an ECDSA private key.

The object can be created randomly, from WIF, from raw 32-byte key material, or from a secret exponent. It can derive the corresponding `PublicKey` and sign Bitcoin messages and transaction digests.

Parameters

- **wif** (*Optional[str]*)
- **secret_exponent** (*Optional[int]*)
- **b** (*Optional[bytes]*)

key

The underlying ECDSA signing key object.

Common methods

``**from_wif(wif)**``

Create an object from WIF or WIFC text.

``**from_bytes(b)**``

Create an object from 32 raw private-key bytes.

``**to_wif(compressed=True)**``

Return the key as WIF or WIFC text.

``**to_bytes()**``

Return the raw 32-byte private key.

``get_public_key()``
Return the corresponding PublicKey.

``sign_message(message, compressed=True)``
Sign a Bitcoin message and return a compact Base64 signature.

``sign_input(tx, txin_index, script, sighash=SIGHASH_ALL)``
Sign a legacy transaction input.

``sign_segwit_input(tx, txin_index, script, amount, sighash=SIGHASH_ALL)``
Sign a SegWit v0 transaction input.

``sign_taproot_input(...)``
Sign a Taproot transaction input by key path or script path.

to_bytes()
Returns key's bytes

Return type
bytes

classmethod from_wif(*wif*)
Creates key from WIFC or WIF format key

Parameters
wif (*str*)

Return type
PrivateKey

classmethod from_bytes(*b*)
Creates a key directly from 32 raw bytes

Parameters
b (*bytes*)

Return type
PrivateKey

to_wif(*compressed=True*)
Returns key in WIFC or WIF string

Pseudocode:

```
network_prefix = (1 byte version number)
data = network_prefix + (32 bytes number/key) [ + 0x01 if compressed ]
data_hash = SHA-256( SHA-256( data ) )
checksum = (first 4 bytes of data_hash)
wif = Base58CheckEncode( data + checksum )
```

Parameters
compressed (*bool*)

Return type
str

sign_message(*message*, *compressed=True*)

Signs the message with the private key (deterministically)

Bitcoin uses a compact format for message signatures (for tx sigs it uses normal DER format). The format has the normal r and s parameters that ECDSA signatures have but also includes a prefix which encodes extra information. Using the prefix the public key can be reconstructed when verifying the signature.

Prefix values:

27 - 0x1B = first key with even y
28 - 0x1C = first key with odd y
29 - 0x1D = second key with even y
30 - 0x1E = second key with odd y

If key is compressed add 4 (31 - 0x1F, 32 - 0x20, 33 - 0x21, 34 - 0x22 respectively)

Returns a Bitcoin compact signature in Base64

Notes

This pure-Python signing path is not side-channel hardened.

Parameters

- **message** (*str*)
- **compressed** (*bool*)

Return type
str | None

sign_input(*tx*, *txin_index*, *script*, *sighash=1*)

Parameters

- **tx** (*Transaction*)
- **txin_index** (*int*)
- **script** (*Script*)
- **sighash** (*int*)

Return type
str

sign_segwit_input(*tx*, *txin_index*, *script*, *amount*, *sighash=1*)

Parameters

- **tx** (*Transaction*)
- **txin_index** (*int*)
- **script** (*Script*)
- **amount** (*int*)
- **sighash** (*int*)

Return type
str

sign_taproot_input(*tx*, *txin_index*, *utxo_scripts*, *amounts*, *script_path=False*, *tapleaf_script=[]*, *tapleaf_scripts=None*, *sighash=0*, *tweak=True*)

Parameters

- **tx** (*Transaction*)
- **txin_index** (*int*)
- **utxo_scripts** (*list[Script]*)
- **amounts** (*list[int]*)
- **script_path** (*bool*)
- **tapleaf_script** (*Script*)
- **tapleaf_scripts** (*Script | List[Script] | List[List[Script]] | bytes | None*)
- **sighash** (*int*)
- **tweak** (*bool*)

Return type
str

get_public_key()

Returns the corresponding PublicKey

Return type
PublicKey

class bitcoinutils.keys.**PublicKey**(*hex_str=None*, *message=None*, *signature=None*)

Bases: object

Represents an ECDSA public key.

Parameters

- **hex_str** (*Optional[str]*)
- **message** (*Optional[str]*)
- **signature** (*Optional[bytes]*)

key

the raw public key of 64 bytes (x, y coordinates of the ECDSA curve)

Type
bytes

from_hex(*hex_str*)

creates an object from a hex string in SEC format (classmethod)

Parameters
hex_str (*str*)

Return type
PublicKey

from_message_signature(*signature*)

NO-OP! (classmethod)

Parameters

- **message** (*str*)
- **signature** (*bytes*)

Return type
PublicKey

verify_message(*address*, *signature*, *message) (classmethod*)

constructs the public key, confirms the address and verifies the signature (classmethod)

Parameters

- **address** (*str*)
- **signature** (*str*)
- **message** (*str*)

Return type
bool

verify(*signature*, *message*)

returns true if the message was signed with this public key's corresponding private key.

Parameters

- **signature** (*str*)
- **message** (*str*)

Return type
bool

to_hex(*compressed=True*)

returns the key as hex string (in SEC format - compressed by default)

Parameters
compressed (*bool*)

Return type
str

to_x_only_hex(*script*)

returns the x coordinate only as hex string before tweaking (needed for taproot)

Return type
str

to_taproot_hex(*script*)

returns the x coordinate only as hex string after tweaking (needed for taproot)

Parameters
scripts (*Script | List[Script] | List[List[Script]] | None*)

Return type
Tuple[str, bool]

is_y_even()

returns true if y coordinate is even

Return type
bool

to_bytes()

returns the key's raw bytes

Return type
bytes

to_hash160()

returns the hash160 hex string of the public key

Parameters
compressed (*bool*)

Return type
str

get_address(*compressed=True)*)

returns the corresponding P2pkhAddress object

Parameters
compressed (*bool*)

Return type
P2pkhAddress

get_segwit_address()

returns the corresponding P2wpkhAddress object

Return type
P2wpkhAddress

get_taproot_address(*scripts*)

returns the corresponding P2trAddress object

Parameters
scripts (*Script | List[Script] | List[List[Script]] | bytes | None*)

Return type
P2trAddress

classmethod from_hex(*hex_str*)

Creates a public key from a hex string (SEC format)

Parameters
hex_str (*str*)

Return type
PublicKey

to_bytes()

Returns key's bytes

Return type
bytes

to_hex(*compressed=True*)

Returns public key as a hex string (SEC format - compressed by default)

Parameters
compressed (*bool*)

Return type
str

`to_x_only_hex()`

Returns the x coordinate of the public key as hex string.

Return type
str

`to_taproot_hex(scripts=None)`

Returns the tweaked x coordinate of the public key as a hex string.

Parameters
scripts (*list[list[Script]]*) – a list of list of Scripts describing the merkle tree of scripts to commit

Return type
Tuple[str, bool]

`is_y_even()`

Returns True if the y coordinate of the public key is even and False otherwise.

Return type
bool

`classmethod from_message_signature(message, signature)`

Recovers a public key from a Bitcoin-signed message and a 65-byte compressed signature.

Parameters

- **message** (*str*)
- **signature** (*bytes*)

Return type
PublicKey

`classmethod verify_message(address, signature, message)`

Creates a public key from a message signature and verifies message

Bitcoin uses a compact format for message signatures (for tx sigs it uses normal DER format). The format has the normal r and s parameters that ECDSA signatures have but also includes a prefix which encodes extra information. Using the prefix the public key can be reconstructed from the signature.

Prefix values:
27 - 0x1B = first key with even y
28 - 0x1C = first key with odd y
29 - 0x1D = second key with even y
30 - 0x1E = second key with odd y

If key is compressed add 4 (31 - 0x1F, 32 - 0x20, 33 - 0x21, 34 - 0x22 respectively)

Raises
ValueError – If signature is invalid

Parameters

- **address** (*str*)
- **signature** (*str*)
- **message** (*str*)

Return type
bool

`verify`(*signature*, *message*)

Verifies that the message was signed with this public key's corresponding private key.

Parameters

- **signature** (*str*)
- **message** (*str*)

Return type
bool

`to_hash160`(*compressed=True*)

Returns the RIPEMD(SHA256( )) of the public key in hex

Parameters
compressed (*bool*)

Return type
str

`get_address`(*compressed=True*)

Returns the corresponding P2PKH Address (default compressed)

Parameters
compressed (*bool*)

Return type
P2pkhAddress

`get_segwit_address`()

Returns the corresponding P2WPKH address

Only compressed is allowed. It is otherwise identical to normal P2PKH address.

Return type
P2wpkhAddress

`get_taproot_address`(*scripts=None*)

Returns the corresponding P2TR address

Only compressed is allowed. Taproot uses x-only public key with even y (02 compressed keys). By default tagged_hashes are used.

scripts contains the list of lists of Scripts describing the merkle tree

Parameters
scripts (*Script | List[Script] | List[List[Script]] | bytes | None*)

Return type
P2trAddress

class bitcoinutils.keys.**Address**(*address=None*, *hash160=None*, *script=None*)

Bases: ABC

Represents a Bitcoin address

hash160

the hash160 string representation of the address; hash160 represents two consequtive hashes of the public key or the redeam script, first a SHA-256 and then an RIPEMD-160

Type
str

`from_address`(*address*)

instantiates an object from address string encoding

Parameters
address (*str*)

Return type
Address

`from_hash160`(*hash160_str*)

instantiates an object from a hash160 hex string

Parameters
hash160 (*str*)

Return type
Address

`from_script`(*redeem_script*)

instantiates an object from a redeem_script

Parameters
script (*Script*)

Return type
Address

`to_string`()

returns the address's string encoding

Return type
str

`to_hash160`()

returns the address's hash160 hex string representation

Return type
str

Raises

- **TypeError** – No parameters passed
- **ValueError** – If an invalid address or hash160 is provided.

Parameters

- **address** (*Optional[str]*)
- **hash160** (*Optional[str]*)
- **script** (*Optional[Script]*)

`classmethod from_address`(*address*)

Creates an address object from an address string

Parameters
address (*str*)

Return type
Address

classmethod from_hash160(*hash160*)

Creates an address object from a hash160 string

Parameters
hash160 (*str*)

Return type
Address

classmethod from_script(*script*)

Creates an address object from a Script object

Parameters
script (*Script*)

Return type
Address

to_hash160()

Returns as hash160 hex string

Return type
str

get_type()

Returns the type of address

Return type
str

to_string()

Returns as address string

Pseudocode:

```
network_prefix = (1 byte version number)
data = network_prefix + hash160_bytes
data_hash = SHA-256( SHA-256( hash160_bytes ) )
checksum = (first 4 bytes of data_hash)
address_bytes = Base58CheckEncode( data + checksum )
```

Return type
str

to_script_pub_key()

Overriden from subclasses

Return type
Script

class bitcoinutils.keys.**P2pkhAddress**(*address=None*, *hash160=None*)

Bases: Address

Encapsulates a P2PKH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)

- **hash160** (*Optional[str]*)

to_script_pub_key()

returns the scriptPubKey (P2PKH) that corresponds to this address

Return type
Script

get_type()

returns the type of address

Return type
str

to_script_pub_key()

Returns the scriptPubKey (P2PKH) that corresponds to this address

Return type
Script

get_type()

Returns the type of address

Return type
str

class bitcoinutils.keys.**P2shAddress**(*address=None*, *hash160=None*, *script=None*)

Bases: Address

Encapsulates a P2SH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **hash160** (*Optional[str]*)
- **script** (*Optional[Script]*)

to_script_pub_key()

returns the scriptPubKey (P2SH) that corresponds to this address

Return type
Script

get_type()

returns the type of address

Return type
str

to_script_pub_key()

Returns the scriptPubKey (P2SH) that corresponds to this address

Return type
Script

get_type()

Returns the type of address

Return type
str

class bitcoinutils.keys.**SegwitAddress**(*address=None*, *witness_program=None*, *script=None*, *version='p2wpkhv0'*)

Bases: ABC

Represents a Bitcoin segwit address

Note that currently the python bech32[m] reference implementation is used (by Pieter Wuille).

witness_program

for segwit v0 this is the hash string representation of either the address; it can be either a public key hash (P2WPKH) or the hash of the script (P2WSH)

for segwit v1 (aka taproot) this is the public key

Type
str

from_address(*address*)

instantiates an object from address string encoding

Parameters
address (*str*)

Return type
SegwitAddress

from_program(*hash_str*)

instantiates an object from a witness program hex string

from_script(*witness_script*)

instantiates an object from a witness_script

Parameters
script (*Script*)

Return type
SegwitAddress

to_string()

returns the address's string encoding (Bech32)

Return type
str

to_hash()

returns the address's hash hex string representation

Raises

- **TypeError** – No parameters passed
- **ValueError** – If an invalid address or hash is provided.

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **script** (*Optional[Script]*)

- **version** (*str*)

classmethod from_address(*address*)

Creates an address object from an address string

Parameters
address (*str*)

Return type
SegwitAddress

classmethod from_witness_program(*witness_program*)

Creates an address object from a hash string

Parameters
witness_program (*str*)

Return type
SegwitAddress

classmethod from_script(*script*)

Creates an address object from a Script object

Parameters
script (*Script*)

Return type
SegwitAddress

to_witness_program()

Returns witness program as hex string

Return type
str

to_string()

Returns as address string

Uses a segwit's python reference implementation for now. (TODO)

Return type
str

to_script_pub_key()

Overriden from subclasses

Return type
Script

class bitcoinutils.keys.**P2wpkhAddress**(*address=None*, *witness_program=None*, *version='p2wpkhv0'*)

Bases: SegwitAddress

Encapsulates a P2WPKH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **version** (*str*)

to_script_pub_key()

returns the scriptPubKey of a P2WPKH witness script

Return type
Script

get_type()

returns the type of address

Return type
str

to_script_pub_key()

Returns the scriptPubKey of a P2WPKH witness script

Return type
Script

get_type()

Returns the type of address

Return type
str

class bitcoinutils.keys.**P2wshAddress**(*address=None*, *witness_program=None*, *script=None*, *version='p2wshv0'*)

Bases: SegwitAddress

Encapsulates a P2WSH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **script** (*Optional[Script]*)
- **version** (*str*)

from_script(*witness_script*)

instantiates an object from a witness_script

get_type()

returns the type of address

Return type
str

to_script_pub_key()

Returns the scriptPubKey of a P2WPKH witness script

Return type
Script

get_type()

Returns the type of address

Return type
str

class bitcoinutils.keys.**P2trAddress**(*address=None*, *witness_program=None*, *version='p2trv1'*, *is_odd=False*)

Bases: SegwitAddress

Encapsulates a P2TR (Taproot) address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **version** (*str*)
- **is_odd** (*bool*)

to_script_pub_key()

returns the scriptPubKey of a P2TR witness script

Return type
Script

get_type()

returns the type of address

Return type
str

to_script_pub_key()

Returns the scriptPubKey of a P2TR witness script

Return type
Script

get_type()

Returns the type of address

Return type
str

is_odd()

Returns True if the y coordinate of the public key is odd and False otherwise.

Return type
bool

2.15 Script API

class bitcoinutils.script.**Script**(*script*)

Represents any script in Bitcoin

A Script contains just a list of OP_CODES and also knows how to serialize into bytes

script

the list with all the script OP_CODES and data

Type
list

to_bytes()

returns a serialized byte version of the script

Return type
bytes

to_hex()

returns a serialized version of the script in hex

Return type
str

get_script()

returns the list of strings that makes up this script

Return type
list[*Any*]

copy()

creates a copy of the object (classmethod)

Parameters
script (*Script*)

Return type
Script

from_raw()

Parameters

- **scriptrawhex** (*str | bytes*)
- **has_segwit** (*bool*)

to_p2sh_script_pub_key()

converts script to p2sh scriptPubKey (locking script)

Return type
Script

to_p2wsh_script_pub_key()

converts script to p2wsh scriptPubKey (locking script)

Return type
Script

Raises
ValueError – If string data is too large or integer is negative

Parameters
script (*list[Any]*)

classmethod copy(*script*)

Deep copy of Script

Parameters
script (*Script*)

Return type
Script

to_bytes()

Converts the script to bytes

If an OP code the appropriate byte is included according to: https://en.bitcoin.it/wiki/Script If not consider it data (signature, public key, public key hash, etc.) and and include with appropriate OP_PUSHDATA OP code plus length

Return type
bytes

to_hex()

Converts the script to hexadecimal

Return type
str

static from_raw(*scriptrawhex*, *has_segwit=False*)

Import a Script commands list from raw hexadecimal data.

Parameters

- **scriptrawhex** (*str or bytes*) – The raw script as hexadecimal text or bytes.
- **has_segwit** (*bool*) – Whether to parse using SegWit-specific script handling.

get_script()

Returns script as array of strings

Return type
list[*Any*]

to_p2sh_script_pub_key()

Converts script to p2sh scriptPubKey (locking script)

Calculates hash160 of the script and uses it to construct a P2SH script.

Return type
Script

to_p2wsh_script_pub_key()

Converts script to p2wsh scriptPubKey (locking script)

Calculates the sha256 of the script and uses it to construct a P2WSH script.

Return type
Script

bitcoinutils.script.**is_p2pkh**(*script*)

Returns True for a P2PKH scriptPubKey.

Parameters
script (*Script*)

Return type
bool

bitcoinutils.script.**is_p2sh**(*script*)

Returns True for OP_HASH160 <20-byte-hash> OP_EQUAL.

Parameters
script (*Script*)

Return type
bool

bitcoinutils.script.**is_p2wpkh**(*script*)

Returns True for OP_0 <20-byte-hash>.

Parameters
script (*Script*)

Return type
bool

bitcoinutils.script.**is_p2wsh**(*script*)

Returns True for OP_0 <32-byte-hash>.

Parameters
script (*Script*)

Return type
bool

bitcoinutils.script.**is_p2tr**(*script*)

Returns True for OP_1 <32-byte-key>.

Parameters
script (*Script*)

Return type
bool

2.16 Transactions API

class bitcoinutils.transactions.**TxInput**(*txid*, *txout_index*, *script_sig=None*, *sequence=b'\xfd\xff\xff\xff'*)

Bases: object

Represents a transaction input.

A transaction input requires a transaction id of a UTXO and the index of that UTXO.

Parameters

- **txid** (*str*)
- **txout_index** (*int*)
- **script_sig** (*Script | None*)
- **sequence** (*str | bytes*)

txid

the transaction id as a hex string (little-endian as displayed by tools)

Type
str

txout_index

the index of the UTXO that we want to spend

Type
int

script_sig

the script that satisfies the locking conditions (aka unlocking script)

Type
list (strings)

sequence

the input sequence (for timelocks, RBF, etc.)

Type
bytes

to_bytes()

serializes TxInput to bytes

Return type
bytes

copy()

creates a copy of the object (classmethod)

Parameters
txin (*TxInput*)

Return type
TxInput

from_raw()

instantiates object from raw hex input (classmethod)

Parameters

- **txinputrawhex** (*str | bytes*)
- **cursor** (*int*)
- **has_segwit** (*bool*)

to_bytes()

Serializes to bytes

Return type
bytes

static from_raw(*txinputrawhex*, *cursor=0*, *has_segwit=False*)

Imports a TxInput from a Transaction's hexadecimal data

Parameters

- **txinputrawhex** (*str | bytes*)
- **cursor** (*int*)
- **has_segwit** (*bool*)

txinputrawhex

The hexadecimal raw string of the Transaction
Type
string (hex)

cursor

The cursor of which the algorithm will start to read the data

Type
int

has_segwit

Is the Tx Input segwit or not

Type
boolean

classmethod copy(*txin*)

Deep copy of TxInput

Parameters
txin (*TxInput*)

Return type
TxInput

class bitcoinutils.transactions.**TxWitnessInput**(*stack*)

Bases: object

A list of the witness items required to satisfy the locking conditions
of a segwit input (aka witness stack).

Parameters
stack (*list[str]*)

stack

the witness items (hex str) list

Type
list

to_bytes()

returns a serialized byte version of the witness items list

Return type
bytes

copy()

creates a copy of the object (classmethod)

Parameters
txwin (*TxWitnessInput*)

Return type
TxWitnessInput

to_bytes()

Converts to bytes

Return type
bytes

classmethod copy(*txwin*)

Deep copy of TxWitnessInput

Parameters
txwin (*TxWitnessInput*)

Return type
TxWitnessInput

class bitcoinutils.transactions.**TxOutput**(*amount*, *script_pubkey*)

Bases: object

Represents a transaction output

Parameters

- **amount** (*int*)
- **script_pubkey** (*Script*)

amount

the value we want to send to this output in satoshis

Type
int

script_pubkey

the script that will lock this amount

Type
Script

to_bytes()

serializes TxInput to bytes

Return type
bytes

copy()

creates a copy of the object (classmethod)

Parameters
txout (*TxOutput*)

Return type
TxOutput

from_raw()

instantiates object from raw hex output (classmethod)

Parameters

- **txoutputrawhex** (*str | bytes*)
- **cursor** (*int*)
- **has_segwit** (*bool*)

to_bytes()

Serializes to bytes

Return type
bytes

static **from_raw**(*txoutputrawhex*, *cursor=0*, *has_segwit=False*)

Imports a TxOutput from a Transaction's hexadecimal data

Parameters

- **txoutputrawhex** (*str | bytes*)

- **cursor** (*int*)
- **has_segwit** (*bool*)

txoutputrawhex

The hexadecimal raw string of the Transaction

Type
string (hex)

cursor

The cursor of which the algorithm will start to read the data

Type
int

has_segwit

Is the Tx Output segwit or not

Type
boolean

classmethod copy(*txout*)

Deep copy of TxOutput

Parameters
txout (*TxOutput*)

Return type
TxOutput

class bitcoinutils.transactions.**Sequence**(*seq_type*, *value*, *is_type_block=True*)

Bases: object

Helps setting up appropriate sequence. Used to provide the sequence to transaction inputs and to scripts.

value

The value of the block height or the 512 seconds increments

Type
int

seq_type

Specifies the type of sequence (TYPE_RELATIVE_TIMELOCK | TYPE_ABSOLUTE_TIMELOCK | TYPE_REPLACE_BY_FEE

Type
int

is_type_block

If type is TYPE_RELATIVE_TIMELOCK then this specifies its type (block height or 512 secs increments)

Type
bool

for_input_sequence()

Serializes the relative sequence as required in a transaction

Return type
str | bytes | None

for_script()

Returns the appropriate integer for a script; e.g. for relative timelocks

Return type
int

Raises
ValueError – if the value is not within range of 2 bytes.

Parameters

- **seq_type** (*int*)
- **value** (*int*)
- **is_type_block** (*bool*)

for_input_sequence()

Creates a relative timelock sequence value as expected from TxInput sequence attribute

Return type
str | bytes | None

for_script()

Creates a relative/absolute timelock sequence value as expected in scripts

Return type
int

class bitcoinutils.transactions.**Locktime**(*value*)

Bases: object

Helps setting up appropriate locktime.

value

The value of the block height or the Unix epoch (seconds from 1 Jan 1970 UTC)

Type
int

for_transaction()

Serializes the locktime as required in a transaction

Return type
bytes

Raises
ValueError – if the value is not within range of 2 bytes.

Parameters
value (*int*)

for_transaction()

Creates a timelock as expected from Transaction

Return type
bytes

class bitcoinutils.transactions.**Transaction**(*inputs=None*, *outputs=None*, *locktime=b'\x00\x00\x00\x00'*, *version=b'\x02\x00\x00\x00'*, *has_segwit=False*, *witnesses=None*)

Bases: object

Represents a Bitcoin transaction.

A transaction contains inputs, outputs, version, locktime, and optionally SegWit witness stacks. It can serialize itself, parse raw transactions, compute txids/wtxids, and produce legacy, SegWit v0, and Taproot signing digests.

Parameters

- **inputs** (*list[TxInput] | None*)
- **outputs** (*list[TxOutput] | None*)
- **locktime** (*str | bytes*)
- **version** (*bytes*)
- **has_segwit** (*bool*)
- **witnesses** (*list[TxWitnessInput] | None*)

inputs

Transaction inputs.

Type
list[TxInput]

outputs

Transaction outputs.

Type
list[TxOutput]

locktime

Transaction locktime.

Type
bytes

version

Transaction version.

Type
bytes

has_segwit

Whether the transaction should serialize with SegWit marker/flag.

Type
bool

witnesses

Witness stacks corresponding to inputs.

Type
list[TxWitnessInput]

Common methods

``from_raw(rawtxhex)``
Instantiate a transaction from serialized raw hexadecimal data.

``to_bytes(has_segwit)``
Serialize the transaction to bytes.

``to_hex()`` / ``serialize()``
Serialize the transaction to hexadecimal text.

``get_txid()``
Return the transaction id without witness data.

``get_wtxid()``
Return the witness transaction id.

``get_size()`` / ``get_vsize()``
Return byte size and virtual size.

``copy(tx)``
Deep-copy a transaction.

``set_witness(txin_index, witness)``
Set the witness stack for an input.

``get_transaction_digest(...)``
Return the legacy signing digest.

``get_transaction_segwit_digest(...)``
Return the SegWit v0 signing digest.

``get_transaction_taproot_digest(...)``
Return the Taproot signing digest.

static from_raw(*rawtxhex*)
Imports a Transaction from hexadecimal data.

Parameters
rawtxhex (*str | bytes*)

rawtxhex
The hexadecimal raw string of the Transaction.
Type
string (hex)

classmethod copy(*tx*)
Deep copy of Transaction

Parameters
tx (*Transaction*)

Return type
Transaction

set_witness(*txin_index*, *witness*)
Safely set a witness at the specified index

Parameters

- **txin_index** (*int*)

- **witness** (*TxWitnessInput*)

get_transaction_digest(*txin_index*, *script*, *sighash=1*)

Returns the transaction's digest for signing. https://en.bitcoin.it/wiki/OP_CHECKSIG

SIGHASH types (see constants.py):
SIGHASH_ALL - signs all inputs and outputs (default)
SIGHASH_NONE - signs all of the inputs
SIGHASH_SINGLE - signs all inputs but only txin_index output
SIGHASH_ANYONECANPAY (only combined with one of the above)
- with ALL - signs all outputs but only txin_index input
- with NONE - signs only the txin_index input
- with SINGLE - signs txin_index input and output

Parameters

- **txin_index** (*int*)
- **script** (*Script*)
- **sighash** (*int*)

txin_index

The index of the input that we wish to sign

Type
int

script

The scriptPubKey of the UTXO that we want to spend

Type
list (string)

sighash

The type of the signature hash to be created

Type
int

get_transaction_segwit_digest(*txin_index*, *script*, *amount*, *sighash=1*)

Returns the segwit v0 transaction's digest for signing. https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki

SIGHASH types (see constants.py):
SIGHASH_ALL - signs all inputs and outputs (default)
SIGHASH_NONE - signs all of the inputs
SIGHASH_SINGLE - signs all inputs but only txin_index output
SIGHASH_ANYONECANPAY (only combined with one of the above)
- with ALL - signs all outputs but only txin_index input
- with NONE - signs only the txin_index input
- with SINGLE - signs txin_index input and output

Parameters

- **txin_index** (*int*) – The index of the input that we wish to sign.
- **script** (*Script*) – The scriptCode that corresponds to the SegWit output being spent.

- **amount** (*int*) – The amount of the UTXO being spent, in satoshis.
- **sighash** (*int*) – The signature hash type to create.

get_transaction_taproot_digest(*txin_index*, *script_pubkeys*, *amounts*, *ext_flag=0*, *script=None*, *leaf_ver=192*, *sighash=0*)

Returns the segwit v1 (taproot) transaction's digest for signing. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki Also consult Bitcoin Core code at: https://github.com/bitcoin/bitcoin/blob/29c36f070618ea5148cd4b2da3732ee4d37af66b/src/script/interpreter.cpp#L1478 And: https://github.com/bitcoin/bitcoin/blob/b5f33ac1f82aea290b4653af36ac2ad1bf1cce7b/test/functional/test_framework/script.py

SIGHASH types (see constants.py):
TAPROOT_SIGHASH_ALL - signs all inputs and outputs (default)
SIGHASH_ALL - signs all inputs and outputs
SIGHASH_NONE - signs all of the inputs
SIGHASH_SINGLE - signs all inputs but only txin_index output
SIGHASH_ANYONECANPAY (only combined with one of the above)
- with ALL - signs all outputs but only txin_index input
- with NONE - signs only the txin_index input
- with SINGLE - signs txin_index input and output

Parameters

- **txin_index** (*int*) – The index of the input that we wish to sign.
- **script_pubkeys** (*list[Script]*) – The scriptPubKeys that correspond to all inputs/UTXOs.
- **amounts** (*list[int]*) – The amounts that correspond to all inputs/UTXOs, in satoshis.
- **ext_flag** (*int*) – Extension mechanism. Use 0 for key path and 1 for script path.
- **script** (*Script, optional*) – The tapleaf script being spent when `ext_flag` is 1.
- **leaf_ver** (*int*) – The script version. Defaults to `LEAF_VERSION_TAPSCRIPT`.
- **sighash** (*int*) – The signature hash type to create.

to_bytes(*has_segwit*)

Serializes to bytes

Parameters
has_segwit (*bool*)

Return type
bytes

get_txid()

Hashes the serialized (bytes) tx to get a unique id

Return type
str

get_wtxid()

Hashes the serialized (bytes) tx including segwit marker and witnesses

Return type
str

`get_size()`

Gets the size of the transaction

Return type
int

`get_vsize()`

Gets the virtual size of the transaction.

For non-segwit txs this is identical to get_size(). For segwit txs the marker and witnesses length needs to be reduced to 1/4 of its original length. Thus it is substructed from size and then it is divided by 4 before added back to size to produce vsize (always rounded up).

https://en.bitcoin.it/wiki/Weight_units

Return type
int

`to_hex()`

Converts object to hexadecimal string

Return type
str

`serialize()`

Converts object to hexadecimal string

Return type
str

`to_psbt()`

Create a PSBT from this transaction.

Returns
A new PSBT wrapping this transaction.

Return type
PSBT

`classmethod from_psbt(psbt_b64_or_hex)`

Extract a signed transaction from a finalized PSBT.

Parameters
psbt_b64_or_hex (*str*) – Base64- or hex-encoded PSBT string.

Returns
The extracted signed transaction.

Return type
Transaction

2.17 HD Wallet API

Minimal BIP-32/BIP-39 HD wallet support.

This module intentionally implements only the functionality used by bitcoinutils: deriving private keys from a mnemonic or an extended private key and returning them as `bitcoinutils.keys.PrivateKey` objects.

class bitcoinutils.hdwallet.**HDWallet**(*xprivate_key=None*, *path=None*, *mnemonic=None*)

Minimal HD wallet wrapper used by bitcoinutils examples and tests.

The wrapper supports deriving Bitcoin private keys from a BIP-39 mnemonic or an extended private key. It does not implement the full external `hdwallet` package API.

Parameters

- **xprivate_key** (*Optional[str]*)
- **path** (*Optional[str]*)
- **mnemonic** (*Optional[str]*)

classmethod from_mnemonic(*mnemonic*)

Instantiate from a BIP-39 mnemonic code.

Parameters
mnemonic (*str*)

classmethod from_xprivate_key(*xprivate_key*, *path=None*)

Instantiate from an extended private key and derivation path.

Parameters

- **xprivate_key** (*str*)
- **path** (*str | None*)

from_path(*path*)

Derive and select a private key from an absolute BIP-32 path.

Parameters
path (*str*)

get_private_key()

Return a PrivateKey object used throughout bitcoinutils.

2.18 Descriptors API

Small fixed-key output descriptor support.

This module implements an educational subset of Bitcoin Core output descriptors. It supports fixed public keys and address/script conversion, but intentionally does not implement ranged descriptors, xpub derivation, private keys, or Miniscript.

The checksum algorithm is ported from Bitcoin Core's MIT-licensed functional test framework (`test/functional/test_framework/descriptors.py`).

exception bitcoinutils.descriptors.**DescriptorError**

Bases: `ValueError`

Raised when a descriptor is malformed or unsupported.

bitcoinutils.descriptors.**descriptor_checksum**(*desc*)

Return the 8-character Bitcoin Core descriptor checksum.

Parameters
desc (*str*)

Return type
str

bitcoinutils.descriptors.**add_descriptor_checksum**(*desc*)

Return *desc* with a Bitcoin Core compatible checksum suffix.

Parameters
desc (*str*)

Return type
str

class bitcoinutils.descriptors.**Descriptor**(*name*, *args*, *checksum=None*)

Bases: object

Parsed fixed-key output descriptor.

Parameters

- **name** (*str*)
- **args** (*tuple[object, ...]*)
- **checksum** (*str | None*)

name: str

args: tuple[object, ...]

checksum: str | None = None

classmethod from_string(*desc*, *require_checksum=False*)

Parameters

- **desc** (*str*)
- **require_checksum** (*bool*)

Return type
Descriptor

to_string(*with_checksum=False*)

Parameters
with_checksum (*bool*)

Return type
str

validate_checksum()

Return type
bool

get_type()

Return type
str

to_script_pub_key()

Return type
Script

`to_address`()

Return type
Address | SegwitAddress

`bitcoinutils.descriptors.`**`parse_descriptor`**(*desc*, *require_checksum=False*)

Parse a fixed-key output descriptor.

Parameters

- **desc** (*str*)
- **require_checksum** (*bool*)

Return type
Descriptor

2.19 PSBT API

BIP-0174 Partially Signed Bitcoin Transaction (PSBT) support.

Implements PSBT version 0 per https://github.com/bitcoin/bips/blob/master/bip-0174.mediawiki

`class bitcoinutils.psbt.`**`PSBTInput`**

Bases: `object`

Data associated with a single PSBT input.

`class bitcoinutils.psbt.`**`PSBTOutput`**

Bases: `object`

Data associated with a single PSBT output.

`class bitcoinutils.psbt.`**`PSBT`**(*tx*)

Bases: `object`

BIP-174 Partially Signed Bitcoin Transaction (version 0).

Parameters
tx (*Transaction*)

tx

The unsigned transaction (no scriptSigs, no segwit flag).

Type
Transaction

inputs

Per-input metadata, partial signatures, etc.

Type
list[PSBTInput]

outputs

Per-output metadata.

Type
list[PSBTOutput]

unknown_global

Unknown global key-value pairs.

Type
dict[bytes, bytes]

to_bytes()

Serialize this PSBT to binary format.

Return type
bytes

to_base64()

Serialize to base64 string.

Return type
str

to_hex()

Serialize to hex string.

Return type
str

classmethod from_bytes(*data*)

Deserialize a PSBT from raw bytes.

Parameters
data (*bytes*)

Return type
PSBT

classmethod from_base64(*b64_str*)

Deserialize a PSBT from a base64 string.

Parameters
b64_str (*str*)

Return type
PSBT

classmethod from_hex(*hex_str*)

Deserialize a PSBT from a hex string.

Parameters
hex_str (*str*)

Return type
PSBT

update_input(*index*, *non_witness_utxo=None*, *witness_utxo=None*, *redeem_script=None*, *witness_script=None*, *sighash_type=None*, *bip32_derivs=None*)

Add UTXO and script metadata for an input.

Parameters

- **index** (*int*)
- **non_witness_utxo** (*Transaction | None*)
- **witness_utxo** (*TxOutput | None*)

- **redeem_script** (*Script | None*)
- **witness_script** (*Script | None*)
- **sighash_type** (*int | None*)
- **bip32_derivs** (*dict[bytes, tuple[bytes, list[int]]] | None*)

update_output(*index*, *redeem_script=None*, *witness_script=None*, *bip32_derivs=None*)

Add script metadata for an output.

Parameters

- **index** (*int*)
- **redeem_script** (*Script | None*)
- **witness_script** (*Script | None*)
- **bip32_derivs** (*dict[bytes, tuple[bytes, list[int]]] | None*)

sign_input(*input_index*, *private_key*, *sighash=1*)

Sign a PSBT input using the library's signing methods.

Parameters

- **input_index** (*int*) – Index of the input to sign.
- **private_key** (*PrivateKey*) – The private key to sign with.
- **sighash** (*int*) – Sighash type (default SIGHASH_ALL).

Returns
True if a signature was produced.

Return type
bool

combine(*other*)

Combine this PSBT with another, returning a new merged PSBT.

Both PSBTs must have the same unsigned transaction.

Parameters
other (*PSBT*)

Return type
PSBT

finalize_input(*input_index*)

Finalize a single input, constructing final scriptSig / witness.

Parameters
input_index (*int*)

finalize()

Finalize all inputs.

extract_transaction()

Extract the fully signed transaction.

Returns
The complete, signed transaction ready for broadcast.

Return type
Transaction

2.20 Block API

class bitcoinutils.block.**BlockHeader**(*version=None*, *previous_block_hash=None*, *merkle_root=None*, *timestamp=None*, *target_bits=None*, *nonce=None*)

Bases: `object`

Represents a Bitcoin block header. This class encapsulates the details of a block's header in the blockchain, which includes essential mining and blockchain continuity information.

Parameters

- **version** (*int | None*)
- **previous_block_hash** (*bytes | None*)
- **merkle_root** (*bytes | None*)
- **timestamp** (*int | None*)
- **target_bits** (*int | None*)
- **nonce** (*int | None*)

version

Version of the block protocol.

Type
Optional[int]

previous_block_hash

256-bit hash of the previous block.

Type
Optional[bytes]

merkle_root

256-bit hash based on all of the transactions in the block.

Type
Optional[bytes]

timestamp

Block creation time in seconds since the Unix epoch.

Type
Optional[int]

target_bits

Encoded difficulty target as a compact format.

Type
Optional[int]

nonce

Arbitrary number that is adjusted to achieve the required proof of work.

Type
Optional[int]

__init__(*version*, *previous_block_hash*, *merkle_root*, *timestamp*, *target_bits*, *nonce*)

Initializes a new instance of the BlockHeader.

from_raw(*rawhexdata*)

Constructs a BlockHeader object from raw block header data.

Parameters
rawhexdata (*str | bytes*)

__str__()

Provides a human-readable representation of the BlockHeader.

__repr__()

Provides a precise string representation of the BlockHeader object that could be used to recreate the object.

get_version()

Retrieves the version of the block.

Return type
int | None

get_previous_block_hash()

Retrieves the hash of the previous block.

Return type
bytes | None

get_merkle_root()

Retrieves the Merkle root of the transactions in the block.

Return type
bytes | None

get_timestamp()

Retrieves the timestamp of when the block was created.

Return type
int | None

get_target_bits()

Retrieves the target bits that encode the mining difficulty target.

Return type
int | None

get_nonce()

Retrieves the nonce used for mining to achieve the block's proof of work.

Return type
int | None

format_timestamp()

Formats the timestamp into a human-readable UTC datetime string.

get_target_hex()

Decodes target bits into the full proof-of-work target.

static from_raw(*rawhexdata*)

Constructs a BlockHeader object from a raw block header data.

Parameters
rawhexdata (*Union[str, bytes]*) – Raw hexadecimal or byte data representing the block header.

Returns
An instance of BlockHeader initialized from the provided raw data.

Return type
BlockHeader

Raises

- **TypeError** – If the input data type is not a string or bytes.
- **ValueError** – If the length of raw data does not match the expected size of a block header.

`get_version()`

Returns the block version, or None if not set.

Return type
int | None

`get_previous_block_hash()`

Returns the previous block hash as bytes, or None if not set.

Return type
bytes | None

`get_merkle_root()`

Returns the merkle root as bytes, or None if not set.

Return type
bytes | None

`get_timestamp()`

Returns the block timestamp, or None if not set.

Return type
int | None

`get_target_bits()`

Returns the compact form of the target difficulty (target_bits), or None if not set.

Return type
int | None

`get_nonce()`

Returns the nonce used for mining, or None if not set.

Return type
int | None

`format_timestamp()`

Formats the block's timestamp into a human-readable UTC datetime string.

Returns
The formatted UTC datetime string representing the block's timestamp.

Return type
str

`get_target_hex()`

Decodes the compact representation of the target bits into the full target hash that a block's hash must be less than or equal to, in order to solve the block.

Returns

The target value as a 64-character hexadecimal string, representing
the full 256-bit target hash used in the proof of work.

Return type
str

`serialize_header()`

Serializes the block header in the format required for hashing.

`get_block_hash()`

Calculates the block hash by double SHA-256 hashing the header.

`class bitcoinutils.block.Block(`*magic=None*, *block_size=None*, *header=None*, *transaction_count=None*, *transactions=None*`)`

Bases: `object`

Represents a Bitcoin block, encapsulating the block's fundamental components including the block header and a list of transactions.

Parameters

- **magic** (*bytes | None*)
- **block_size** (*int | None*)
- **header** (*BlockHeader | None*)
- **transaction_count** (*int | None*)
- **transactions** (*list[Transaction] | None*)

`magic`

Magic value indicating the network for which the block was intended.

Type
Optional[bytes]

`block_size`

Size of the block in bytes.

Type
Optional[int]

`header`

Header information of the block including version, previous block hash, etc.

Type
Optional[BlockHeader]

`transaction_count`

Number of transactions included in the block.

Type
Optional[int]

`transactions`

A list of transactions contained in the block.

Type
Optional[list[Transaction]]

__init__(*magic*, *block_size*, *header*, *transaction_count*, *transactions*)

Initializes a new instance of the Block class with specified attributes.

from_raw(*rawhexdata*)

Constructs a Block object from a raw block data. Raises TypeError if the input is neither bytes nor a hexadecimal string. Raises ValueError if the length of raw data does not match the expected size.

Parameters
rawhexdata (*str | bytes*)

__str__()

Provides a human-readable representation of the Block, showing all attributes.

__repr__()

Provides a precise string representation of the Block object that could be used to recreate the object.

get_magic_bytes()

Retrieves the block's magic bytes as a tuple containing the hexadecimal representation and a descriptive string. Raises ValueError if the magic bytes are not set.

Return type
tuple[str, str]

get_block_size()

Retrieves the size of the block in bytes.

Return type
int

get_block_header()

Retrieves the BlockHeader object associated with the block. Raises ValueError if the block header is not set.

Return type
BlockHeader

get_transactions()

Retrieves the list of Transaction objects contained in the block. Raises ValueError if there are no transactions.

Return type
list[*Transaction*]

get_transactions_count()

Retrieves the count of transactions contained in the block. Raises ValueError if there are no transactions.

Return type
int

get_coinbase_transaction()

Retrieves the first transaction in the block, typically the coinbase transaction. Raises ValueError if there are no transactions.

Return type
Transaction

get_block_reward()

Calculates the total output amount of the coinbase transaction, representing the block reward.

Return type
int

`get_witness_transactions()`

Returns a list of transactions that include SegWit data.

Return type
list[*Transaction*]

`get_legacy_transactions()`

Returns a list of transactions that are non-SegWit, legacy-style transactions.

Return type
list[*Transaction*]

`static from_raw(rawhexdata)`

Constructs a Block instance from raw block data in hexadecimal or byte format.

Parameters
rawhexdata (*Union[str, bytes]*) – The raw data of the block in hexadecimal or bytes format.

Returns
A fully parsed Block object.

Return type
Block

Raises

- **TypeError** – If the input is not a string or bytes.
- **ValueError** – If the input does not meet the expected block structure or size.

`get_magic_bytes()`

Retrieves the magic bytes and its corresponding network description.

Returns
The hexadecimal representation of the magic bytes and its description.

Return type
tuple[str, str]

Raises
ValueError – If the magic bytes are not set.

`get_block_size()`

Retrieves the size of the block in bytes.

Returns
The size of the block.

Return type
int

`get_block_header()`

Retrieves the header of the block.

Returns
The block header object.

Return type
BlockHeader

Raises
ValueError – If the block header is not set.

`get_transactions()`

Retrieves the transactions contained in the block.

Returns
A list of transactions.

Return type
list[Transaction]

Raises
ValueError – If there are no transactions in the block.

`get_transactions_count()`

Retrieves the number of transactions in the block.

Returns
The number of transactions.

Return type
int

Raises
ValueError – If there are no transactions.

`get_coinbase_transaction()`

Retrieves the coinbase transaction from the block, which is the first transaction in the block and contains the block reward.

Returns
The coinbase transaction.

Return type
Transaction

Raises
ValueError – If there are no transactions in the block.

`get_block_reward()`

Calculates the total amount of the block reward by summing the outputs of the coinbase transaction.

Returns
The total output amount in the coinbase transaction, representing the block reward.

Return type
int

Raises
ValueError – If there are no transactions in the block.

`get_witness_transactions()`

Return transactions that contain SegWit data.

Returns
Transactions with SegWit data.

Return type
list[Transaction]

`get_legacy_transactions()`

Return legacy transactions.

Returns
Transactions without SegWit data.

Return type
list[Transaction]

2.21 Utilities API

class bitcoinutils.utils.**Secp256k1Params**

class bitcoinutils.utils.**ControlBlock**(*pubkey*, *scripts*, *index*, *is_odd=False*)

Represents a control block for spending a taproot script path

Parameters

- **pubkey** (*PublicKey*)
- **scripts** (*None | list[list[Script]]*)
- **index** (*int*)

pubkey

the internal public key object

Type
PublicKey

scripts

a list of list of Scripts describing the merkle tree of scripts to commit

Type
list[list[Script]]

merkle_path

the pre-calculated merkle path

Type
bytes

to_bytes()

returns the control block as bytes

Return type
bytes

to_hex()

returns the control block as a hexadecimal string

Return type
str

to_bytes()

Return type
bytes

to_hex()

Converts object to hexadecimal string

Returns
The control block as a hexadecimal string

Return type
str

bitcoinutils.utils.**get_tag_hashed_merkle_root**(*scripts*)

Tag hashed merkle root of all scripts - tag hashes tapleafs and branches as needed.

Scripts is a list of list of Scripts describing the merkle tree of scripts to commit Example of scripts' list: [[A, B], C]

Parameters
scripts (*None | Script | list[Script] | list[list[Script]]*)

Return type
bytes

bitcoinutils.utils.**to_satoshis**(*num*)

Converts from any number type (int/float/Decimal) to satoshis (int)

Parameters
num (*int | float | Decimal*)

bitcoinutils.utils.**prepend_compact_size**(*data*)

Counts bytes and returns them with their varint (or compact size) prepended.

Parameters
data (*bytes*)

Return type
bytes

bitcoinutils.utils.**encode_varint**(*i*)

Encode a potentially very large integer into varint bytes. The length should be specified in little-endian.

https://bitcoin.org/en/developer-reference#compactsize-unsigned-integers

Parameters
i (*int*)

Return type
bytes

bitcoinutils.utils.**parse_compact_size**(*data*)

Parse variable integer. Returns (count, size)

Parameters
data (*bytes*)

Return type
tuple

bitcoinutils.utils.**get_transaction_length**(*data*)

Return length of a transaction, including handling for SegWit transactions.

Parameters
data (*bytes*)

Return type
int

bitcoinutils.utils.**is_address_bech32**(*address*)

Returns if an address (string) is bech32 or not

Parameters
address (*str*) – The address to check

Returns
True if the address is bech32, False otherwise

Return type
bool

bitcoinutils.utils.**vi_to_int**(*byteint*)

Converts varint bytes to int

Parameters
byteint (*bytes*)

Return type
Tuple[int, int]

bitcoinutils.utils.**add_magic_prefix**(*message*)

Required prefix when signing a message

Parameters
message (*str*)

Return type
bytes

bitcoinutils.utils.**tagged_hash**(*data*, *tag*)

Return a BIP-340 style tagged hash.

Tagged hashes ensure that hashes used in one context cannot be used in another. A tagged hash is `SHA256(SHA256(tag) || SHA256(tag) || data)`.

Parameters

- **data** (*bytes*)
- **tag** (*str*)

Return type
bytes

bitcoinutils.utils.**calculate_tweak**(*pubkey*, *scripts*)

Calculates the tweak to apply to the public and private key when required.

Parameters

- **pubkey** (*PublicKey*)
- **scripts** (*None | Script | list[Script] | list[list[Script]] | bytes*)

Return type
int

bitcoinutils.utils.**tapleaf_tagged_hash**(*script*)

Calculates the tagged hash for a tapleaf

Parameters
script (*Script*) – The script to calculate the tagged hash for

Returns
The tagged hash of the tapleaf

Return type
bytes

bitcoinutils.utils.**tapbranch_tagged_hash**(*thashed_a*, *thashed_b*)

Calculates the tagged hash for a tapbranch

Parameters

- **thashed_a** (*bytes*) – First tagged hash
- **thashed_b** (*bytes*) – Second tagged hash

Returns
The tagged hash of the tapbranch

Return type
bytes

bitcoinutils.utils.**negate_privkey**(*key*)

Negate private key, if necessary

Parameters
key (*bytes*)

Return type
str

bitcoinutils.utils.**tweak_taproot_pubkey**(*internal_pubkey*, *tweak*)

Tweaks the public key with the specified tweak. Required to create the taproot public key from the internal key.

Parameters

- **internal_pubkey** (*bytes*)
- **tweak** (*int*)

Return type
Tuple[bytes, bool]

bitcoinutils.utils.**tweak_taproot_privkey**(*privkey*, *tweak*)

Tweaks the private key before signing with it. Check if public key's y is even and negate the private key before tweaking if it is not.

Parameters

- **privkey** (*bytes*)
- **tweak** (*int*)

Return type
bytes

bitcoinutils.utils.**b_to_h**(*b*)

Converts bytes to hexadecimal string

Parameters
b (*bytes*)

Return type
str

bitcoinutils.utils.**h_to_b**(*h*)

Converts hexadecimal string to bytes

Parameters
h (*str*)

Return type
bytes

bitcoinutils.utils.**h_to_i**(*hex_str*)

Converts a string hexadecimal to a number

Parameters
hex_str (*str*)

Return type
int

bitcoinutils.utils.**i_to_h64**(*i*)

Converts an int to a string hexadecimal (padded to 64 hex chars)

Parameters
i (*int*)

Return type
str

bitcoinutils.utils.**b_to_i**(*b*)

Converts a bytes to a number

Parameters
b (*bytes*)

Return type
int

bitcoinutils.utils.**i_to_b32**(*i*)

Converts a integer to bytes

Parameters
i (*int*)

Return type
bytes

bitcoinutils.utils.**i_to_b**(*i*)

Converts a integer to bytes

Parameters
i (*int*)

Return type
bytes

2.22 Proxy API

exception bitcoinutils.proxy.**RPCException**(*rpc_error*)

Bases: Exception

Raised when the Bitcoin Core JSON-RPC interface returns an error.

Parameters
rpc_error (*Dict[str, Any]*)

Return type
None

code

The JSON-RPC error code (e.g. `-32600` for invalid request).

Type
int | None

message

Human-readable description returned by Bitcoin Core.

Type
str | None

error

The raw error dict from the JSON-RPC response.

Type
dict

class bitcoinutils.proxy.**NodeProxy**(*rpcuser*, *rpcpassword*, *host=None*, *port=None*, *timeout=30*, *use_https=False*, *max_retries=3*, *backoff_base=0.5*)

Bases: `object`

JSON-RPC proxy for a Bitcoin Core node.

Supports any RPC command via dynamic attribute access:

```
proxy = NodeProxy("rpcuser", "rpcpassword")
height = proxy.getblockcount()
block_hash = proxy.getblockhash(height)
```

Parameters

- **rpcuser** (*str*) – RPC username as defined in `bitcoin.conf`.
- **rpcpassword** (*str*) – RPC password as defined in `bitcoin.conf`.
- **host** (*str, optional*) – Host where the Bitcoin node resides (default `127.0.0.1`).
- **port** (*int, optional*) – Port to connect to. Uses the default port for the active network (set via `bitcoinutils.setup.setup()`).
- **timeout** (*int, optional*) – HTTP timeout in seconds (default 30).
- **use_https** (*bool, optional*) – If *True* use HTTPS; otherwise plain HTTP (default *False*).
- **max_retries** (*int, optional*) – Maximum number of retries on retriable connection errors (default 3). Set to 0 to disable retries.
- **backoff_base** (*float, optional*) – Base delay in seconds for exponential back-off between retries (default 0.5). The *n*-th retry waits `backoff_base * 2^(n-1)`.

Raises
ValueError – If *rpcuser* or *rpcpassword* is empty / not provided.

batch_(*rpc_calls*)

Send a batch JSON-RPC request.

Parameters
rpc_calls (*list of lists*) – Each inner list is `["method", param1, param2, ...]`.

Returns
The results of each call, in the same order.

Return type
list

Raises
RPCException – If any individual call returns an error.

CHAPTER

THREE

INDICES AND TABLES

- genindex
- modindex
- search